

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

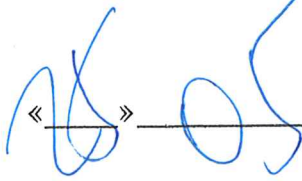
Отчет о выполнении курсовой работы
по дисциплине «Теоретические основы баз данных»
Реализация базы данных для предметной области
«Ветеринарная клиника»

Выполнил студент группы
№5130201/30001

Проверил преподаватель

Радионова П. В. 

Попов С. Г. 

«» 2025г.

1/3 19.03.25 06.05.25
2/3
Санкт-Петербург, 2025

Реферат

Курсовая работа посвящена разработке базы данных для предметной области «Функционирование работы многопрофильной ветеринарной клиники», которая включает учёт ветеринаров, администраторов, клиентов и их питомцев, записи на приёмы, медицинские назначения и покупки в зоомагазине. Основная проблема заключалась в необходимости эффективного хранения и быстрого доступа к данным о животных, их владельцах и оказанных услугах. В ходе работы была проанализирована предметная область, спроектирована реляционная база данных, создана и заполнена база данных, а также был написан код для 8 запросов, сформированных на основе созданной базы данных. База данных была реализована в командной строке MySQL, взаимодействие с базой осуществлялось через консоль Windows. Для заполнения базы данных был написан программный код на языке Python. Полученная система может быть внедрена в ветеринарных клиниках для автоматизации учёта пациентов и упрощения работы администраторов.

Содержание

Введение	4
1 Постановка задачи	5
2 Исследование процессов работы ветеринарной клиники	6
2.1 Описание предметной области	6
2.2 Цели создания базы данных	8
2.3 Описание сущностей	8
2.4 Описание ролей	9
2.5 ER-диаграмма	11
2.6 Чтение ER-диаграммы	11
2.7 Схема связи объектов	11
3 Проектирование	12
3.1 Таблицы с описанием таблиц баз данных	12
3.2 Лингвистическое описание схемы	17
3.3 Графическое описание схемы базы данных на русском языке	20
3.4 Графическое описание схемы базы данных на английском языке	21
3.5 Создание базы данных и её заполнение	21
4 Программирование	22
4.1 Запрос 1.1	24
4.2 Запрос 1.2	26
4.3 Запрос 2	27
4.4 Запрос 3	30
4.5 Запрос 4.1	33
4.6 Запрос 4.2	35
4.7 Запрос 5	35
4.8 Запрос 6	38
4.9 Запрос 7	40
4.10 Запрос 8	41
Заключение	46
Список использованных источников	47
Приложение А «Исходный код программы генерации схемы базы данных»	48
Приложение Б «Исходный код программы заполнения базы данных»	51

Введение

Современные организации ежедневно сталкиваются с необходимостью управления большими объемами информации. В условиях ручной обработки информации возрастают риски ошибок, потери данных и неэффективного использования времени сотрудников. Особенно остро эта проблема проявляется в сферах, где важны точность и оперативность обработки данных.

Система баз данных – ключевой инструмент для структурирования информации и её обработки. Базу данных можно рассматривать как хранилище для значительно большего объема информации, занесенного в компьютер.

Базы данных заменяют бумажные архивы и обеспечивают мгновенный доступ к данным. Они ускоряют процесс выполнения обычных операций, минимизируют дублирование и позволяют нескольким пользователям работать с информацией одновременно. Это может быть полезно для отслеживания множественных процессов работы ветеринарной клиники.

Основой взаимодействия с такими системами являются запросы на языке SQL. Запросы дают возможность гибко управлять данными: добавлять, удалять, сортировать и анализировать их в реальном времени. Язык SQL реализует выборку данных по заданным критериям независимо от обновления информации, хранящейся в базе данных.

При этом эффективное проектирование базы данных требует тщательного анализа предметной области и оптимизации структуры таблиц для минимизации избыточности информации. Грамотно спроектированная база данных не только обеспечивает целостность и безопасность данных, но и значительно повышает производительность работы всей информационной системы в целом.

1 Постановка задачи

Цель: реализовать базу данных для предметной области «Функционирование работы ветеринарной клиники».

Для достижения поставленной цели было выделено несколько задач.

1. Составление описания предметной области и формулировка целей создания базы данных.
2. Создание ER-диаграммы.
3. Проектирование базы данных и её лингвистического и графического описания.
4. Создание базы данных.
5. Заполнение базы данных 200-300 тысячами значений.
6. Написание 8 запросов на языке SQL.
7. Генерация 3 гистограмм.

2 Исследование процессов работы ветеринарной клиники

2.1 Описание предметной области

Ветеринарная клиника — специализированное медицинское учреждение, которое оказывает врачебную помощь и обеспечивает уход за домашними животными.

Раннее ветеринария рассматривалась как отрасль общей медицины, однако сейчас она является областью научных знаний, предметом изучения которой является лечение болезней животных, производство безопасных для человека сельскохозяйственных продуктов и защита людей от зоонозных болезней — заболеваний, общих для человека и животных.

Активный рост ветеринарной медицины в России был тесно связан с историческими потребностями. Вспышки инфекционных заболеваний среди животных, особенно лошадей, потребовали вмешательства: лошади играли важную роль в армии, транспорте и сельском хозяйстве, и их здоровье было необходимо для привычного функционирования государства [1]. Параллельно, развитие животноводства требовало квалифицированных специалистов для лечения сельскохозяйственного скота. Обеспечение здоровья скота напрямую влияло на продовольственные запасы. Таким образом, ветеринария стала неотъемлемой частью сельского хозяйства и национальной безопасности.

Со второй половины XX века произошел значительный сдвиг в отношении человека к животным. Некоторые виды животных человек стал воспринимать не только в плоскости служебной или экономической ценности, но и как компаньонов для жизни. Сейчас все больше людей видят в своём питомце объект тёплого общения и внимания, способный давать хозяину радость и эмоции. Вторая половина 60-х годов XX века стала отправной точкой для формирования нового социально-культурного явления — признания за животными-компаньонами определенных прав и потребностей. Возрастающая роль животных в жизни людей, особенно в качестве компаньонов, привела к активному развитию «зооиндустрии». К ней относятся услуги передержки, груминга, производство специализированных кормов, игрушек, одежды и аксессуаров. Это же и стало причиной возникновения ветеринарии мелких домашних животных (ВМДЖ) — области ветеринарной медицины, занимающаяся лечением и профилактикой заболеваний у животных-компаньонов: кошек, собак, попугаев, грызунов и многих других питомцев, содержащихся в квартирах. К середине 1980-х годов в Советском Союзе появилась отдельная

специализация ветеринарных врачей, занимающихся лечением исключительно кошек и собак. В конце 1990-х — начале 2000-х годов в Москве и Санкт-Петербурге появились первые клиники со стационарным лечением, широким спектром аппаратной диагностики для животных. В этих центрах врачи уже имели специализацию.

В настоящие дни работа крупных центров, специализирующихся на помощи мелким домашним животным, не отличается от таковых в гуманитарной медицине [2]. Существуют экспертного уровня ветеринарные офтальмологи, кардиологи, хирурги, эндокринологи, дерматологи, стоматологи, аллергологи и специалисты всех других медицинских направлений. В клиниках для животных способны выполнять сложнейшие операции и диагностические исследования: ЭКГ, УЗИ, компьютерная томография, рентгенография, лабораторные анализы и многие другие. В последние годы все большее значение приобретает превентивная медицина, направленная на профилактику заболеваний с помощью вакцинации, правильного питания и регулярных осмотров.

Ветеринарные клиники также предлагают дополнительные услуги, включая ветеринарную аптеку, где можно приобрести необходимые лекарственные препараты, специальные корма и средства по уходу за питомцами. В случае, когда помочь животному уже невозможно, ветеринарные клиники предлагают услуги эвтаназии и кремации.

Работа ветеринарных клиник в целом похожа на работу обыкновенных частных клиник. Владелец животного может обратиться в любой понравившийся ему ветеринарный центр, зарегистрировать своего питомца и записать его на прием. Ветеринарный врач осмотрит животное, соберет анамнез, поставит первичный диагноз, даст свои комментарии и рекомендации по уходу и лечению и может даже дать направление к другому специалисту: например, кардиологу или аллергологу. После первого же приема у питомца появится своя медицинская карточка в этой клинике, в которой будет храниться вся основная информация о животном, история его обращений, результаты анализов и исследований, а также сведения о проведенных вакцинациях и лечении. Это позволяет ветеринарным врачам отслеживать состояние здоровья животного и оказывать ему наиболее эффективную и своевременную помощь.

В будущем можно ожидать дальнейшего развития ветеринарной медицины: уже сейчас производится трансплантация органов, активно развивается протезирование для животных. После травм и операций осуществляется реабилитация для домашних питомцев, например, физиотерапия и гидротерапия. Все это делает ветеринарию не только наукой о лечении, но и повышении качества жизни домашних животных.

Несмотря на активный рост развития ветеринарной медицины в России и большую потребность в этом среди населения, государство не осуществляет участие в контроле вопросов, связанных с содержанием животных [3]. На данный момент отсутствуют стандарты оказания медицинской помощи домашним питомцам, система лицензирования ветеринарных клиник и сертификации врачей. Это создает определенные риски для владельцев животных, так как на рынке могут появляться неквалифицированные специалисты и недобросовестные клиники. Отсутствие финансирования государством ветеринарных услуг приводит к высокой стоимости лечения для владельцев животных.

Таким образом, ветеринарная медицина прошла долгий путь развития – от надзора за заразными болезнями скота до высокотехнологичных центров, предлагающих полный спектр услуг для домашних питомцев. Однако в заключение стоит подчеркнуть, что здоровье домашнего питомца – это всегда ответственность его владельца.

2.2 Цели создания базы данных

1. Хранение клиентской карточки хозяина животного.
2. Хранение медицинской карточки питомца.
3. Хранение информации о персонале.
4. Хранение информации о проданных товарах.

2.3 Описание сущностей

1. Ветеринарная клиника – медицинское учреждение, специализирующееся на оказании врачебной помощи домашним животным.

Атрибуты: адрес, номер телефона.

2. Администратор – специалист, общающийся с клиентами клиники, координирующий расписание приемов и осуществляющий продажу лекарственных препаратов, кормов и других ветеринарных средств.

Атрибуты: ФИО, номер телефона, адрес электронной почты.

3. Ветеринар – врач, специализирующийся на уходе и лечении мелких домашних животных.

Атрибуты: ФИО, специализация, квалификация, адрес проживания, адрес электронной почты, номер телефона.

4. Медицинский приём – комплекс мероприятий, включающий в себя сбор анамнеза, осмотр, проведение диагностических процедур или медицинских услуг, постановку диагноза и назначение лечения.

Атрибуты: статус приёма, причина обращения, предоставленная услуга, заключение, назначение.

5. Питомец – домашнее животное, имеющее владельца.

Атрибуты: кличка, дата рождения, пол, вид, порода, номер чипа, аллергия.

6. Владелец питомца – человек, содержащий у себя дома животное.

Атрибуты: ФИО, номер телефона, адрес электронной почты, адрес проживания.

7. Товар – ветеринарное средство, продаваемое в ветеринарной клинике.

Атрибуты: наименование, страна производства, поставщик, цена.

2.4 Описание ролей

Администратор является работником ветеринарной клиники и играет роль связующего звена между владельцем питомца и ветеринарным врачом. Он общается с клиентами, осуществляет их запись к врачу, напоминает им о приемах и осуществляет оплату оказанных медицинских услуг. Администратор также производит продажу лекарственных препаратов, кормов и других ветеринарных средств, вплоть до игрушек для животных.

Владелец питомца обращается в ветеринарную клинику очно или онлайн и через администратора записывает своего питомца на прием. Он оплачивает оказанные питомцу медицинские услуги и покупает товары в ветеринарной аптеке.

Ветеринар является лечащим специалистом ветеринарной клиники и проводит медицинские приемы согласно своему расписанию. Он собирает анамнез, проводит осмотр, оказывает медицинские услуги, ставит диагноз и дает назначения и рекомендации к лечению.

2.5 ER-диаграмма

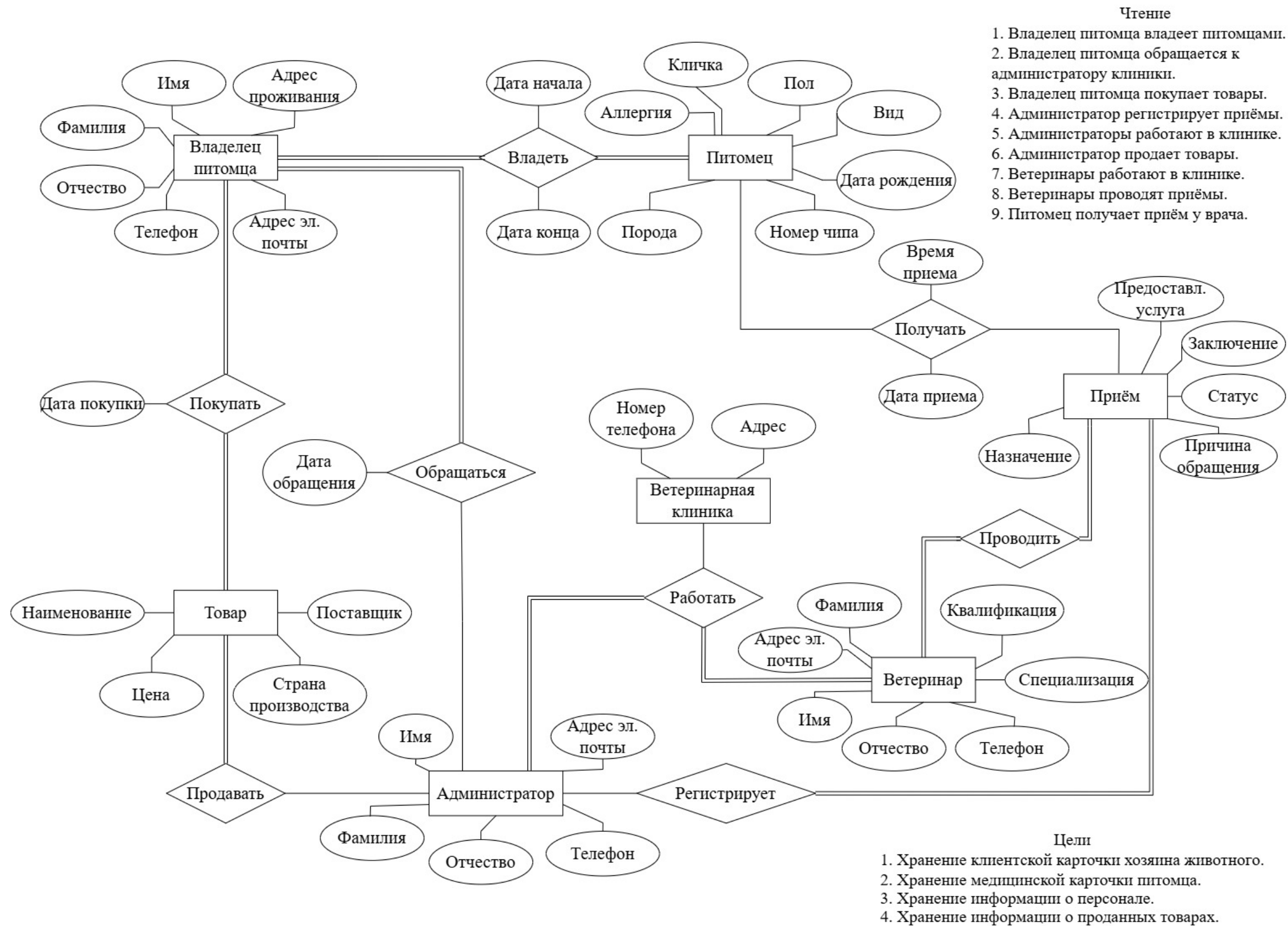


Рис. 1. ER-диаграмма

2.6 Чтение ER-диаграммы

ER-диаграмма приведена на рис. 1.

1. Владелец питомца владеет питомцами.
2. Владелец питомца обращается к администратору клиники.
3. Владелец питомца покупает товары.
4. Администратор регистрирует приёмы.
5. Администраторы работают в клинике.
6. Администратор продает товары.
7. Ветеринары работают в клинике.
8. Ветеринары проводят приёмы.
9. Питомец получает приём у врача.

2.7 Схема связи объектов

На рис. 2 представлена схема связи объектов базы данных.

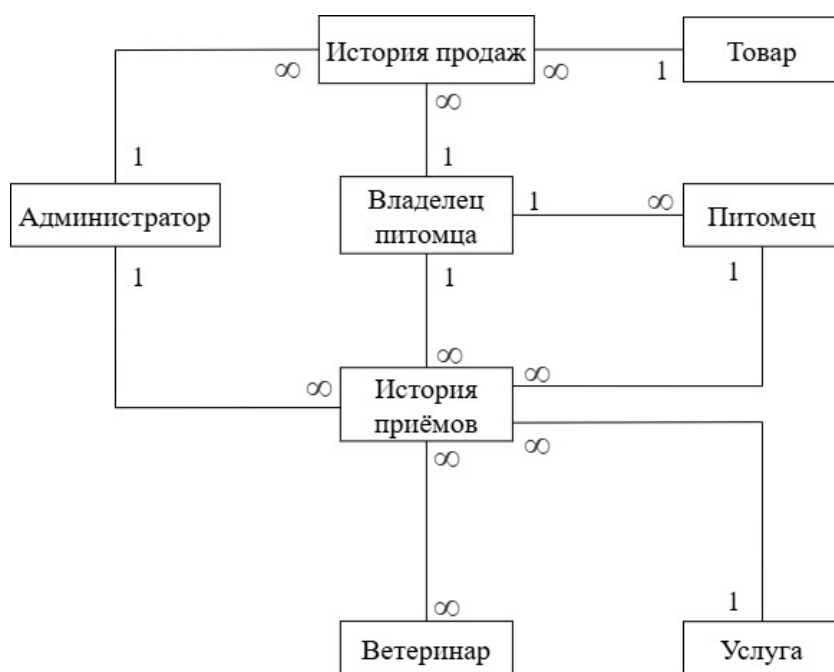


Рис. 2. Схема связи объектов

3 Проектирование

3.1 Таблицы с описанием таблиц баз данных

В таблицах 1-12 описаны атрибуты каждой таблицы проектируемой базы данных. В таблицах перечислены: признак ключа, если поле является ключом, имя атрибута на русском и на английском языках, тип атрибута, размер переменной, значение атрибута по умолчанию. Для внешних ключей также указаны ссылки на таблицы и атрибуты первичных ключей.

Ограничение в 14 символов для атрибута «Вид_название» в таблице 1 обусловлено тем, что это максимальное количество символов среди всех имеющихся в предметной области видов мелких домашних животных.

Таблица 1. Вид - species

Вид - species						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Вид_id	species_id	INT	NOT NULL	-
2	-	Вид_название	name	VARCHAR(14)	NOT NULL	-

Ограничение в 40 символов для атрибута «Порода_название» в таблице 2 обусловлено количеством символов в самом длинном названии породы в предметной области.

Таблица 2. Порода - breed

Порода - breed						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Порода_id	breed_id	INT	NOT NULL	-
2	-	Порода_название	name	VARCHAR(40)	NOT NULL	-
3	FK	Вид_id	species_id	INT	NOT NULL	species(species_id)

Ограничение в 50 символов для атрибутов «Владелец_имя», «Владелец_фамилия» и «Владелец_отчество» в таблице 3 обусловлено тем, что в среднем их длина в выбранной библиотеке Python.Faker не превышает 47 символов.

Ограничение в 20 символов для атрибута «Владелец_номер_телефона» в таблице 3 установлено для возможности использовать символы-разделители, такие как скобка и дефис, совместно с 11 цифрами номера телефона.

Ограничение в 254 символа для атрибута «Владелец_адрес_эл_почты» в таблице 3 обусловлено тем, что максимальная длина адреса электронной почты составляет 254 символа.

Ограничение в 100 символов для атрибута «Владелец_адрес_проживания» в таблице 3 обусловлен тем, что самый длинный адрес из списка содержит 96 символов.

Таблица 3. Владелец - owner

Владелец - owner						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Владелец_id	vet_id	INT	NOT NULL	-
2	-	Владелец_имя	first_name	VARCHAR(50)	NOT NULL	-
3	-	Владелец_фамилия	second_name	VARCHAR(50)	NOT NULL	-
4	-	Владелец_отчество	middle_name	VARCHAR(50)	NULL	-
5	-	Владелец_номер_телефона	phone_number	VARCHAR(20)	NOT NULL	-
6	-	Владелец_адрес_эл_почты	email	VARCHAR(254)	NULL	-
7	-	Владелец_адрес_проживания	address	VARCHAR(100)	NULL	-

Ограничение в 50 символов для атрибута «Питомец_кличка» в таблице 4 установлено потому, что в среднем самые длинные клички выставочных животных не превышают 50 символов.

Ограничение в 7 символов для атрибута «Питомец_пол» в таблице 4 обусловлено максимальным количеством символов среди возможных значений «Male» и «Female».

Ограничение в 100 символов для атрибута «Питомец_аллергия» в таблице 4 установлено потому, что у питомца может быть несколько аллергий, объединенных в одну строку и перечисленных через запятую или точку с запятой.

Таблица 4. Питомец - pet

Питомец - pet						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Питомец_id	vet_id	INT	NOT NULL	-
2	-	Питомец_кличка	nickname	VARCHAR(50)	NOT NULL	-
3	-	Питомец_дата_рождения	birth_date	DATE	NULL	-
4	-	Питомец_пол	sex	VARCHAR(7)	NOT NULL	-
5	-	Питомец_аллергия	allergy	VARCHAR(100)	NULL	-
6	-	Питомец_номер_чипа	chip_number	INT	NULL	-
7	FK	Владелец_id	owner_id	INT	NOT NULL	owner(owner_id)
8	FK	Вид_id	species_id	INT	NOT NULL	species(species_id)
9	FK	Порода_id	breed_id	INT	NULL	breed(breed_id)

Ограничение в 30 символов для атрибута «Услуга_название» в таблице 5 обусловлено тем, что самое длинное название в таблице составляет 28 символов.

Ограничение в 10 символов для атрибута «Услуга_цена» в таблице 5 обусловлено тем, что максимальная стоимость из списка («10 000») состоит из 6 символов.

Таблица 5. Услуга - service

Услуга - service						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Услуга_id	service_id	INT	NOT NULL	-
2	-	Услуга_название	name	VARCHAR(30)	NOT NULL	-
3	-	Услуга_цена	price	DECIMAL(10, 2)	NOT NULL	-

Ограничение в 50 символов для атрибутов «Администратор_имя», «Администратор_фамилия» и «Администратор_отчество» в таблице 6 обусловлено тем, что в среднем их длина в выбранной библиотеке Python.Faker не превышает 47 символов.

Ограничение в 20 символов для атрибута «Администратор_номер_телефона» в таблице 6 установлено для возможности использовать символы-разделители, такие как скобка и дефис, совместно с 11 цифрами номера телефона.

Ограничение в 254 символа для атрибута «Администратор_адрес_эл-почты» в таблице 6 обусловлено тем, что максимальная длина адреса электронной почты составляет 254 символа.

Таблица 6. Администратор - administrator

Администратор - administrator						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Администратор_id	admin_id	INT	NOT NULL	-
2	-	Администратор_имя	first_name	VARCHAR(50)	NOT NULL	-
3	-	Администратор_фамилия	second_name	VARCHAR(50)	NOT NULL	-
4	-	Администратор_отчество	middle_name	VARCHAR(50)	NOT NULL	-
5	-	Администратор_номер_телефона	phone_number	VARCHAR(20)	NOT NULL	-
6	-	Администратор_адрес_эл_почты	email	VARCHAR(254)	NOT NULL	-

Ограничение в 10 символов для атрибута «Прием_статус» в таблице 7 установлен потому, что максимальная длина среди возможных значений: «Completed», «Cancelled», «No-show» – составляет 9 символов.

Ограничение в 100 символов для атрибута «Прием_причина_обращения» в таблице 7 установлено потому, что может быть названо несколько

ко причин, объединенных в одну строку и перечисленных через запятую или точку с запятой.

Ограничение в 200 символов для атрибута «Прием_заключение» в таблице 7 установлено потому, что ветеринар может поставить сразу несколько диагнозов, которые будут объединены в одну строку и перечислены через запятую или точку с запятой.

Ограничение в 200 символов для атрибута «Прием_назначение» в таблице 7 установлено потому, что ветеринар может назначить лечение, состоящее из нескольких рекомендаций, которые будут объединены в одну строку и перечислены через запятую или точку с запятой.

Таблица 7. Прием - appointment

Прием - appointment						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Прием_id	appointment_id	INT	NOT NULL	-
2	-	Прием_статус	status	VARCHAR(10)	NOT NULL	-
3	-	Прием_дата	appointment_date	DATE	NULL	-
4	-	Прием_время	appointment_time	TIME	NOT NULL	-
5	-	Прием_причина_обращения	reason	VARCHAR(100)	NOT NULL	-
6	-	Прием_заключение	conclusion	VARCHAR(200)	NULL	-
7	-	Прием_назначение	treatment	VARCHAR(200)	NULL	-
8	FK	Услуга_id	service_id	INT	NOT NULL	service(service_id)
9	FK	Питомец_id	pet_id	INT	NOT NULL	pet(pet_id)
10	FK	Владелец_id	owner_id	INT	NOT NULL	owner(owner_id)
11	FK	Администратор_id	admin_id	INT	NOT NULL	admin(admin_id)

Ограничение в 50 символов для атрибутов «Ветеринар_имя», «Ветеринар_фамилия» и «Ветеринар_отчество» в таблице 8 обусловлено тем, что в среднем их длина в выбранной библиотеке Python.Faker не превышает 47 символов.

Ограничение в 20 символов для атрибута «Ветеринар_номер_телефона» в таблице 8 установлено для возможности использовать символы-разделители, такие как скобка и дефис, совместно с 11 цифрами номера телефона.

Ограничение в 254 символа для атрибута «Ветеринар_адрес_эл_почты» в таблице 8 обусловлено тем, что максимальная длина адреса электронной почты составляет 254 символа.

Ограничение в 15 символов атрибута «Ветеринар_специализация» в таблице 8 обусловлено тем, что слово максимальной длины среди возможных значений предметной области содержит 11 символов.

Ограничение в 6 символов атрибута «Ветеринар_квалификация» в таблице 8 обусловлено тем, что все возможные значения атрибута: «высшая», «первая», «вторая» – содержат в себе 6 символов.

Таблица 8. Ветеринар - veterinarians

Ветеринар - veterinarian						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Ветеринар_id	vet_id	INT	NOT NULL	-
2	-	Ветеринар_имя	first_name	VARCHAR(50)	NOT NULL	-
3	-	Ветеринар_фамилия	second_name	VARCHAR(50)	NOT NULL	-
4	-	Ветеринар_отчество	middle_name	VARCHAR(50)	NOT NULL	-
5	-	Ветеринар_номер_телефона	phone_number	VARCHAR(20)	NOT NULL	-
6	-	Ветеринар_адрес_эл_почты	email	VARCHAR(254)	NOT NULL	-
7	-	Ветеринар_специализация	specialization	VARCHAR(15)	NOT NULL	-
8	-	Ветеринар_квалификация	qualification	VARCHAR(6)	NOT NULL	-

Таблица 9. Приемы и ветеринары - veterinarian and appointment

Приемы и ветеринары - veterinarian and appointment						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Ветеринар_прием_id	vet_app_id	INT	NOT NULL	-
2	FK	Прием_id	appointment_id	INT	NOT NULL	appointment(appointment_id)
3	FK	Ветеринар_id	vet_id	INT	NOT NULL	veterinarian(vet_id)

Ограничение в 50 символов у атрибута «Поставщик_имя» в таблице 10 обусловлено тем, что в среднем длина названий компаний в выбранной библиотеке Python.Faker не превышает 50 символов.

Таблица 10. Поставщик - supplier

Поставщик - supplier						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Поставщик_id	supplier_id	INT	NOT NULL	-
2	-	Поставщик_имя	name	VARCHAR(50)	NOT NULL	-

Ограничение в 50 символов у атрибута «Товар_наименование» в таблице 11 обусловлено тем, что название товара может содержать в себе некоторые характеристики товара, такие как цвет или вкус.

Ограничение в 20 символов у атрибута «Товар_страна_производства» в таблице 11 обусловлено тем, что очень длинные названия стран можно сократить до аббревиатуры, и в среднем названия стран не содержат в себе более 20 символов.

Ограничение в 10 символов для атрибута «Товар_цена» в таблице 11 обусловлено тем, что максимальная стоимость из списка состоит из 6 символов.

Таблица 11. Товар - product

Товар - product						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Товар_id	product_id	INT	NOT NULL	-
2	-	Товар_наименование	name	VARCHAR(50)	NOT NULL	-
3	-	Товар_страна_производства	country	VARCHAR(20)	NOT NULL	-
2	-	Товар_цена	price	DECIMAL(10, 2)	NOT NULL	-
3	FK	Поставщик_id	supplier_id	INT	NOT NULL	supplier(supplier_id)

Таблица 12. Покупка - purchase

Покупка - purchase						
№	Тип ключа	Название RU	Название EN	Тип атрибута	NULL/NOT NULL	Ссылка
1	PK	Покупка_id	product_id	INT	NOT NULL	-
2	-	Покупка_дата	purchase_date	DATE	NOT NULL	-
3	FK	Администратор_id	admin_id	INT	NOT NULL	administrator(admin_id)
2	FK	Владелец_id	owner_id	INT	NOT NULL	owner(owner_id)
3	FK	Товар_id	product_id	INT	NOT NULL	product(product_id)

3.2 Лингвистическое описание схемы

1. Вид (Вид_id (INT), Вид_название (VARCHAR(14))).
2. Порода (Порода_id (INT), Порода_название (VARCHAR(40)), Вид_id (INT)).
3. Владелец (Владелец_id (INT), Владелец_имя (VARCHAR(50)), Владелец_фамилия (VARCHAR(50)), Владелец_отчество (VARCHAR(50)), Владелец_номер_телефона (VARCHAR(20)), Владелец_адрес_эл_почты (VARCHAR(254)), Владелец_адрес_проживания (VARCHAR(100))).
4. Питомец (Питомец_id (INT), Питомец_кличка (VARCHAR(50)), Питомец_дата_рождения (DATE), Питомец_пол (VARCHAR(7)), Питомец_аллергия (VARCHAR(100)), Питомец_номер_чипа (INT), Владелец_id (INT), Вид_id (INT), Порода_id (INT)).
5. Услуга (Услуга_id (INT), Услуга_название (VARCHAR(30)), Услуга_цена (DECIMAL(10, 2))).
6. Администратор (Администратор_id (INT), Администратор_имя (VARCHAR(50)), Администратор_фамилия (VARCHAR(50)), Администратор_отчество (VARCHAR(50)), Администратор_номер_телефо-

на (VARCHAR(20)), Администратор_адрес_эл_почты (VARCHAR(254))).

7. Прием (Прием_id (INT), Прием_статус (VARCHAR(10)), Прием_дата (DATE), Прием_время (TIME), Прием_причина_обращения (VARCHAR(100)), Прием_заключение (VARCHAR(200)), Прием_назначение (VARCHAR(200)), Услуга_id (INT), Питомец_id (INT), Владелец_id (INT), Администратор_id (INT)).
8. Ветеринар (Ветеринар_id (INT), Ветеринар_имя (VARCHAR(50)), Ветеринар_фамилия (VARCHAR(50)), Ветеринар_отчество (VARCHAR(50)), Ветеринар_номер_телефона (VARCHAR(20)), Ветеринар_адрес_эл_почты (VARCHAR(254)), Ветеринар_специализация (VARCHAR(15)), Ветеринар_квалификация (VARCHAR(6))).
9. Приемы и ветеринары (Ветеринар_прием_id (INT), Прием_id (INT), Ветеринар_id (INT)).
10. Поставщик (Поставщик_id (INT), Поставщик_имя (VARCHAR(50))).
11. Товар (Товар_id (INT), Товар_наименование (VARCHAR(50)), Товар_страна_производства (VARCHAR(20)), Товар_цена (DECIMAL(10, 2)), Поставщик_id (INT)).
12. Покупка (Покупка_id (INT), Покупка_дата (DATE), Администратор_id (INT), Владелец_id (INT), Товар_id (INT)).

Графические описания схемы базы данных на русском и английском языках представлены на рис. 3 и 4 соответственно.

3.3 Графическое описание схемы базы данных на русском языке

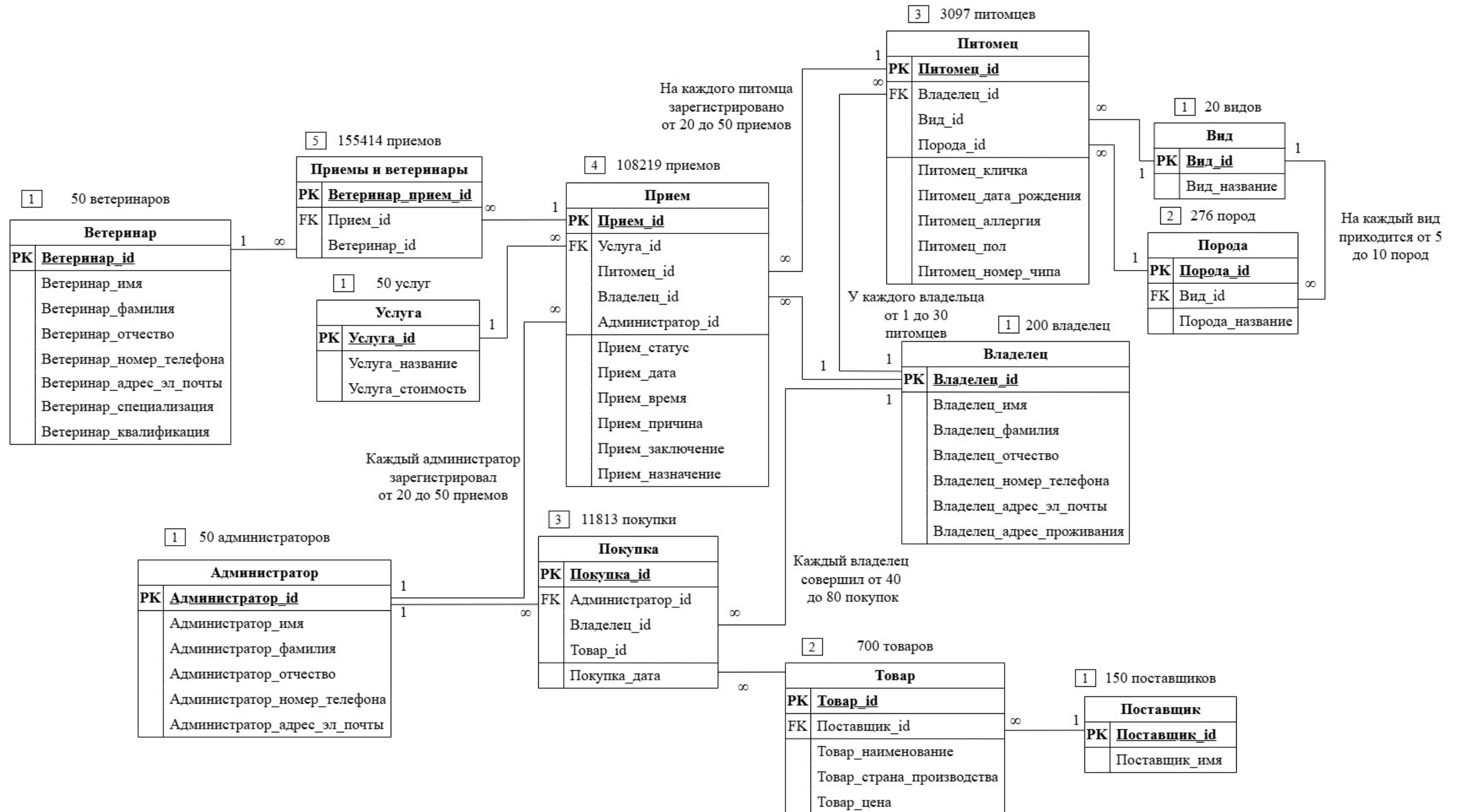


Рис. 3. Схема базы данных на русском языке

3.4 Графическое описание схемы базы данных на английском языке

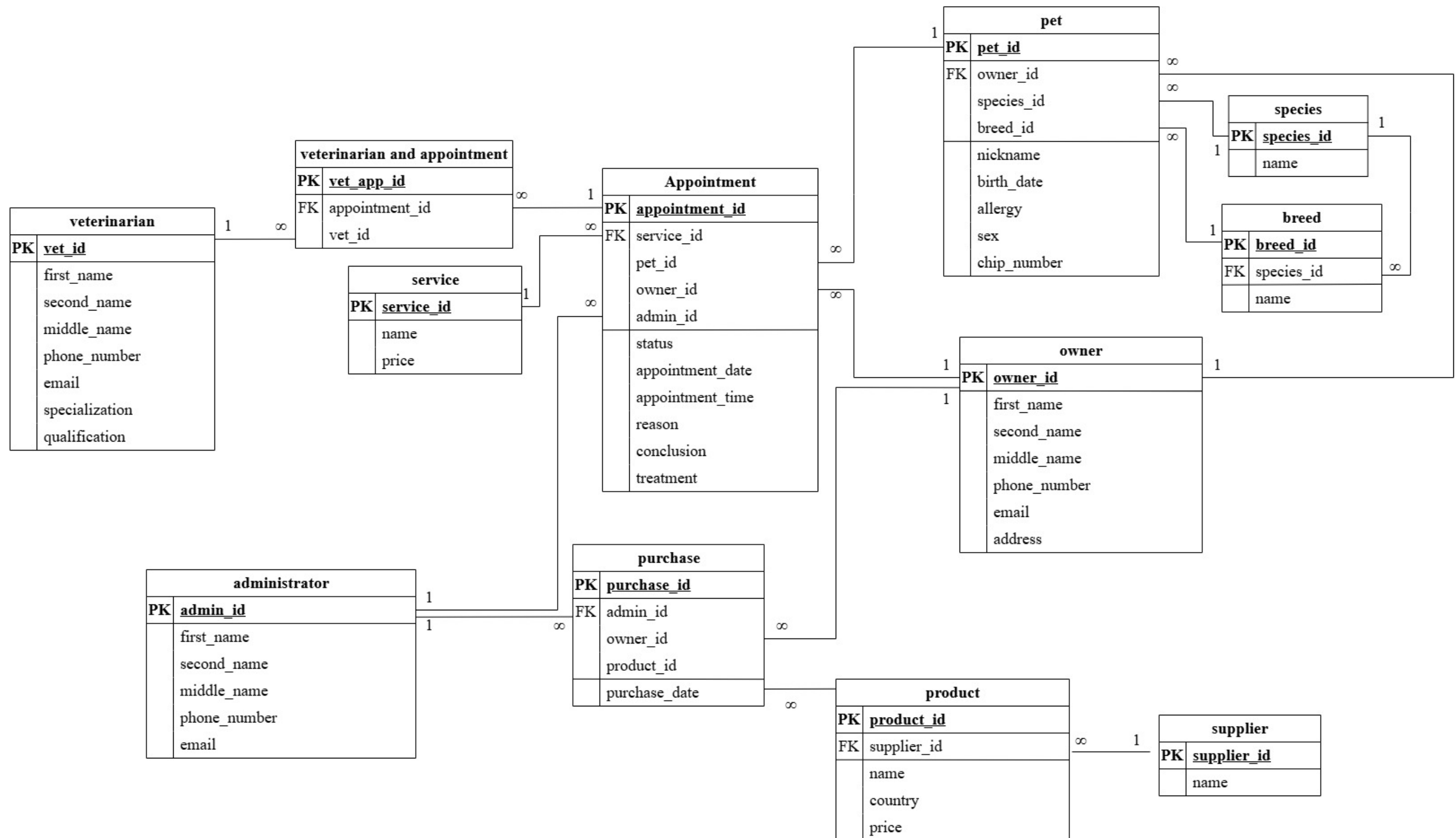


Рис. 4. Схема базы данных на английском языке

3.5 Создание базы данных и её заполнение

База данных и содержащиеся в ней таблицы были созданы с помощью SQL-запросов и командной строки MySQL Command Line Client [5]. Исходный код программы создания базы данных приведен в приложении [Приложение А «Исходный код программы генерации схемы базы данных»](#). Графическое представление схемы базы данных на русском и английском языках представлены на рис. 3, 4.

Для заполнения таблиц базы данных была написана программа на языке Python, её исходный текст приведен в приложении [Приложение Б «Исходный код программы заполнения базы данных»](#). Был установлен автономный драйвер mysql.connector для взаимодействия с серверами MySQL [4]. Была применена настройка API для подключения к базе данных, в конфигурационном файле были указаны хост, имя пользователя, пароль и имя базы данных. Для выполнения SQL-инструкций использовался курсор – экземпляр класса MySQLConnection, наследуемый от MySQLCursor. Вставка данных в таблицы производилась с помощью SQL-команд INSERT, перед вставкой данные из таблиц удалялись с помощью команды TRUNCATE. Часть полей таблицы заполнялась с помощью библиотеки Faker – это пакет Python, который генерирует поддельные данные.

Всего было выполнено 12 запросов для заполнения 12 таблиц, которые содержат 280039 записей. Распределение записей по таблицам представлено в таблице 13.

Таблица 13. Количество записей в таблицах

№	Название таблицы	Количество записей в таблице
1	Вид - species	20
2	Порода - breed	276
3	Владелец - owners	200
4	Питомец - pet	3097
5	Услуга - service	50
6	Администратор - administrator	50
7	Прием - appointment	108219
8	Ветеринар - veterinarian	50
9	Приемы и ветеринары - veterinarian and appointment	155414
10	Поставщик - supplier	150
11	Товар - product	700
12	Покупка - purchase	11813

4 Программирование

Лист проверки выполнения запросов
курсовой работы по дисциплине
«Теоретические основы баз данных» 4 семестр

Имя, отчество, фамилия Радионова Полина Викторовна Группа 5130201/3000 2

Тема работы Вет. клиника

№ п.п.	текст запроса	принято	
		запрос	отчет
1.1	И-ти вет-ра, к-й имеет собак породы А и к-й владелицу к-й покупал товар В	☒	☐
1.2	И-ти [...], товар купила имеет все месяцы 2013г.	☒	☐
2	И-ти кто посетил-т число приемов пт-ца А, администраторам к-го был Б и его имеет вет-р С.	☒	☐
3	Для кажд. укл. посчитать число приемов и число питомцев. Подумать 2D и 3D (1мб.)	☒	☐
4.1	И-ти товары, к-ые покупались ранее	☒	☐
4.2	всего. + к-ые покупались ранее всего.	☒	☐
5	Посчитать число владелиц, с одинаковым числом питомцев + 2D и 3D.	☒	☐
6	И-ти админ. В, к-ой оформили данные покупки товаров, им админ. А	☒	☐
7	И-ти влад-ца, к-й никогда не обр к вет-ру А.	☒	☐
8	Для кажд. вида собаки и первых 10 вет-в посчитать число приемов. 3D и 4D	☒	☐
9		☐	☐
10		☐	☐

УН070

left join
dewer

3-4 FROM
UNION
SELECT
LEFT
JOIN
WHERE
EXIST

4.1 Запрос 1.1

Формулировка запроса

Найти ветеринаров, которые лечили собак породы А, владельцы которых покупали товар В.

Реляционная формула запроса

$$\delta \left(\pi \begin{pmatrix} v.vet_id, \\ v.first_name, \\ v.second_name, \\ v.middle_name \end{pmatrix} \right)$$

$\sigma(s.name = 'Собака' \text{ and } b.name = 'Пудель' \text{ and } prod.name = 'Гигиена \text{ салфетки пасть}')$

$veterinarian \bowtie (veterinarian.vet_id = va.vet_id) veterinarian_appointment$
 $veterinarian_appointment \bowtie (va.appointment_id = a.appointment_id)$
 $appointment$
 $appointment \bowtie (a.pet_id = p.pet_id) pet$
 $pet \bowtie (pet.breed_id = b.breed_id) breed$
 $breed \bowtie (b.species_id = s.species_id) species$
 $pet \bowtie (p.owner_id = o.owner_id) owners$
 $owners \bowtie (o.owner_id = pur.owner_id) purchase$
 $purchase \bowtie (pur.product_id = prod.product_id) product$

Код запроса

```
1  SELECT DISTINCT
2      v.vet_id,
3      v.first_name,
4      v.second_name,
5      v.middle_name
6  FROM
7      veterinarian v
8  JOIN veterinarian_appointment va ON v.vet_id = va.vet_id
9  JOIN appointment a ON va.appointment_id = a.appointment_id
10 JOIN pet p ON a.pet_id = p.pet_id
11 JOIN breed b ON p.breed_id = b.breed_id
12 JOIN species s ON b.species_id = s.species_id
13 JOIN owners o ON p.owner_id = o.owner_id
```

```

14 JOIN purchase pur ON o.owner_id = pur.owner_id
15 JOIN product prod ON pur.product_id = prod.product_id
16 WHERE
17     s.name = 'Собака'
18     AND b.name = 'Пудель'
19     AND prod.name = 'Гигиена салфетки пасть';

```

Результаты запроса

В результате выполнения запроса было выведено 46 строк. Время выполнения запроса составило 0.047 секунд. Результат запроса представлен на рис. 5. Табличное и графическое представления последовательности выполнения запроса представлены на рис. 6 и 7 соответственно.

	vet_id	first_name	second_name	middle_name
▶	32	Тамара	Самойлова	Викторович
	45	Каллистрат	Никонова	Ильич
	3	Леонтий	Орлова	Филиппович
	21	Антонина	Зимина	Рубеновна
	36	Ульяна	Суворова	Абрамович
	50	Руслан	Боброва	Святославовна
	42	Феврония	Муравьев	Макаровна
	24	Алевтина	Лаврентьев	Жанович
	11	Аверкий	Кириллов	Рубеновна
	12	Трофим	Исаев	Васильевна

Рис. 5. Результат выполнения запроса 1.1 в виде фрагмента таблицы

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s		ALL	PRIMARY				20	10.00	Using where; Using temporary
1	SIMPLE	b		ref	PRIMARY,species_id	species_id	4	vetclinic.s.species_id	13	10.00	Using where
1	SIMPLE	p		ref	PRIMARY,owner_id,breed_id	breed_id	5	vetclinic.b.breed_id	11	100.00	
1	SIMPLE	o		eq_ref	PRIMARY	PRIMARY	4	vetclinic.p.owner_id	1	100.00	Using index
1	SIMPLE	pur		ref	product_id,owner_id	owner_id	4	vetclinic.p.owner_id	57	100.00	
1	SIMPLE	prod		eq_ref	PRIMARY	PRIMARY	4	vetclinic.pur.product_id	1	10.00	Using where
1	SIMPLE	a		ref	PRIMARY,pet_id	pet_id	4	vetclinic.p.pet_id	34	100.00	Using index
1	SIMPLE	va		ref	appointment_id,vet_id	appointment_id	4	vetclinic.a.appointment_id	1	100.00	
1	SIMPLE	v		eq_ref	PRIMARY	PRIMARY	4	vetclinic.va.vet_id	1	100.00	

Рис. 6. Табличное представление последовательности выполнения запроса 1.1

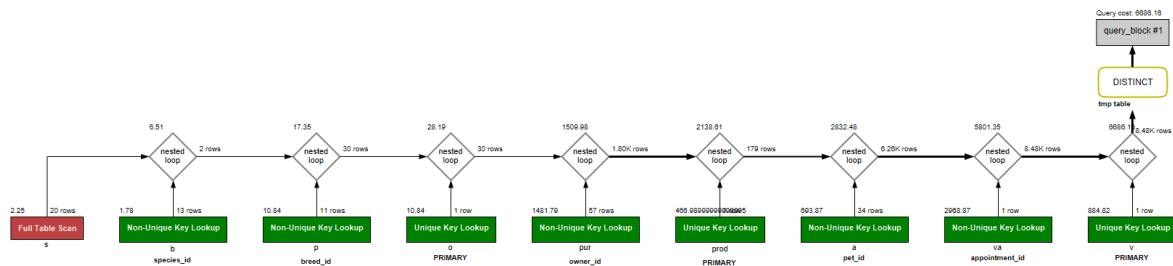


Рис. 7. Графическое представление последовательности выполнения запроса 1.1

4.2 Запрос 1.2

Формулировка запроса

Найти ветеринаров, которые не менее 3-х раз лечили собак породы А, владельцы которых покупали товар В.

Код запроса

```

1  SELECT
2      v.vet_id,
3      v.first_name,
4      v.second_name,
5      v.middle_name,
6      COUNT(*) as treatment_count
7  FROM
8      veterinarian v
9  JOIN veterinarian_appointment va ON v.vet_id = va.vet_id
10 JOIN appointment a ON va.appointment_id = a.appointment_id
11 JOIN pet p ON a.pet_id = p.pet_id
12 JOIN breed b ON p.breed_id = b.breed_id
13 JOIN species s ON b.species_id = s.species_id
14 JOIN owners o ON p.owner_id = o.owner_id
15 JOIN purchase pur ON o.owner_id = pur.owner_id
16 JOIN product prod ON pur.product_id = prod.product_id
17 WHERE
18     s.name = 'Собака'
19     AND b.name = 'Пудель'
20     AND prod.name = 'Гигиена салфетки пасть'
21 GROUP BY
22     v.vet_id, v.first_name, v.second_name, v.middle_name

```

23 **HAVING COUNT(*)** >= 3;

Текстовое объяснение запроса

Запрос с помощью JOIN выполняет соединение 10 таблиц по следующим ключам:

- veterinarian.vet_id → veterinarian_appointment.vet_id;
- veterinarian_appointment.appointment_id → appointment.appointment_id;
- appointment.pet_id → pet.pet_id;
- pet.breed_id → breed.breed_id;
- breed.species_id → species.species_id;
- pet.owner_id → owners.owner_id;
- owners.owner_id → purchase.owner_id;
- purchase.product_id → product.product_id.

Далее с помощью WHERE выбираются записи, соответствующие указанным значениям для species.name, breed.name и product.name. Полученные строки группируются по ветеринарам с помощью GROUP BY veterinarian.vet_id, veterinarian.first_name, veterinarian.second_name, veterinarian.middle_name, чтобы избежать дублирования строк. В конце с помощью фильтрации HAVING COUNT(*) >= 3 остаются только подходящие ветеринары.

Результаты запроса

В результате выполнения запроса была выведена 21 строка. Время выполнения запроса составило 0.047 секунд. Результат запроса представлен на рис. 8. Табличное и графическое представления последовательно выполнения запроса представлены на рис. 9 и 10 соответственно.

4.3 Запрос 2

Формулировка запроса

Посчитать число приемов питомца А, администратором которого был администратор В и ветеринаром которого был ветеринар С.

Код запроса

```
1 SELECT
2     COUNT(*) AS appointment_count
3 FROM
4     appointment a
5 JOIN pet p ON a.pet_id = p.pet_id
6 JOIN administrator adm ON a.admin_id = adm.admin_id
7 JOIN veterinarian_appointment va ON a.appointment_id =
   ↪ va.appointment_id
8 JOIN veterinarian v ON va.vet_id = v.vet_id
9 WHERE
10     p.pet_id = 1
11     AND adm.second_name = 'Рожкова'
12     AND v.second_name = 'Рогова';
```

Текстовое объяснение запроса

С помощью JOIN запрос соединяет таблицы appointment, pet, administrator, veterinarian_appointment и veterinarian по следующим ключам:

- appointment.pet_id → pet.pet_id;
- appointment.admin_id → administrator.admin_id;
- appointment.appointment_id → veterinarian_appointment.appointment_id;
- veterinarian_appointment.vet_id → veterinarian.vet_id.

Далее с помощью WHERE выбираются только записи, соответствующие указанным значениям для pet.pet_id, administrator.second_name и veterinarian.second_name. С использованием COUNT(*) подсчитывается общее количество приемов appointment_count, удовлетворяющих заданным условиям.

Результаты запроса

В результате выполнения запроса была выведена 1 строка. Время выполнения запроса составило 0.016 секунд. Результат запроса представлен на рис. 11. Табличное и графическое представления последовательности выполнения запроса представлены на рис. 12 и 13 соответственно.

	appointment_count
▶	1

Рис. 11. Результат выполнения запроса 2 в виде таблицы

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	p		const	PRIMARY	PRIMARY	4	const	1	100.00	Using index
1	SIMPLE	a		ref	PRIMARY,pet_id,admin_id	pet_id	4	const	29	100.00	
1	SIMPLE	adm		eq_ref	PRIMARY	PRIMARY	4	vetclinic.a.admin_id	1	10.00	Using where
1	SIMPLE	va		ref	appointment_id,vet_id	appointment_id	4	vetclinic.a.appointment_id	1	100.00	
1	SIMPLE	v		eq_ref	PRIMARY	PRIMARY	4	vetclinic.va.vet_id	1	10.00	Using where

Рис. 12. Табличное представление последовательности выполнения запроса 2

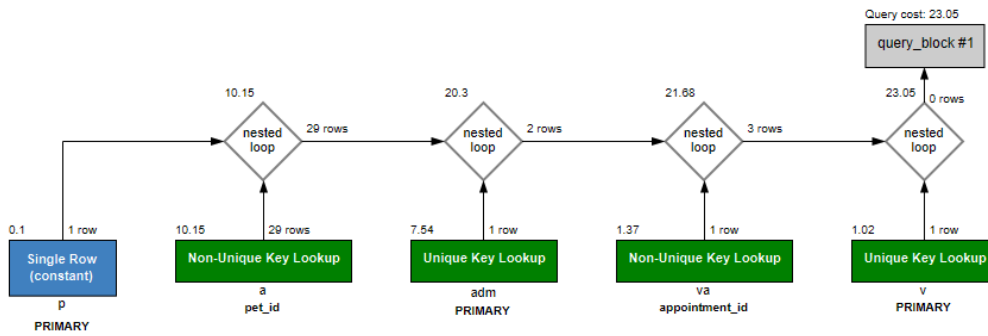


Рис. 13. Графическое представление последовательности выполнения запроса 2

4.4 Запрос 3

Формулировка запроса

Для каждой услуги посчитать число приемов и число питомцев.

Реляционная формула запроса

$$\pi \left(\begin{array}{c} s.service_id, \\ \beta(s.name \text{ AS } service_name), \\ \beta(COUNT(a.appointment_id) \text{ AS } total_appointments), \\ \beta(COUNT(DISTINCT a.pet_id) \text{ AS } unique_pets) \end{array} \right)$$

service $\bowtie$ (service.service_id=appointment.service_id) appointment
 $\gamma(service.service_id, service.name)$

$\tau(\text{service.service_id})$

Код запроса

```
1 SELECT
2     s.service_id,
3     s.name AS service_name,
4     COUNT(a.appointment_id) AS total_appointments,
5     COUNT(DISTINCT a.pet_id) AS unique_pets
6 FROM
7     service s
8 LEFT JOIN
9     appointment a ON s.service_id = a.service_id
10 GROUP BY
11     s.service_id, s.name
12 ORDER BY
13     s.service_id;
```

Результаты запроса

В результате выполнения запроса было выведено 50 строк. Время выполнения запроса составило 0.469 секунд. Результат запроса представлен на рис. 14. Табличное и графическое представления последовательности выполнения запроса представлены на рис. 15 и 16 соответственно.

	service_id	service_name	total_appointments	unique_pets
►	1	Первичный осмотр	2111	1516
	2	Повторный осмотр	2193	1553
	3	Вакцинация	2190	1564
	4	Чипирование	2115	1501
	5	Кастрация кота	2171	1534
	6	Стерилизация кошки	2092	1499
	7	УЗИ брюшной полости	2144	1557
	8	Рентген	2139	1532
	9	Анализ крови	2177	1571
	10	Чистка зубов	2214	1558

Рис. 14. Результат выполнения запроса 3 в виде фрагмента таблицы

Гистограмма запроса

На рис. 17 представлена 2D-гистограмма, демонстрирующая распределение приемов и уникальных питомцев по всем имеющимся в таблице услугам.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	s		ALL					50	100.00	Using filesort
1	SIMPLE	a		ref	service_id	service_id	4	vetclinic.s.service_id	2374	100.00	

Рис. 15. Табличное представление последовательности выполнения запроса 3

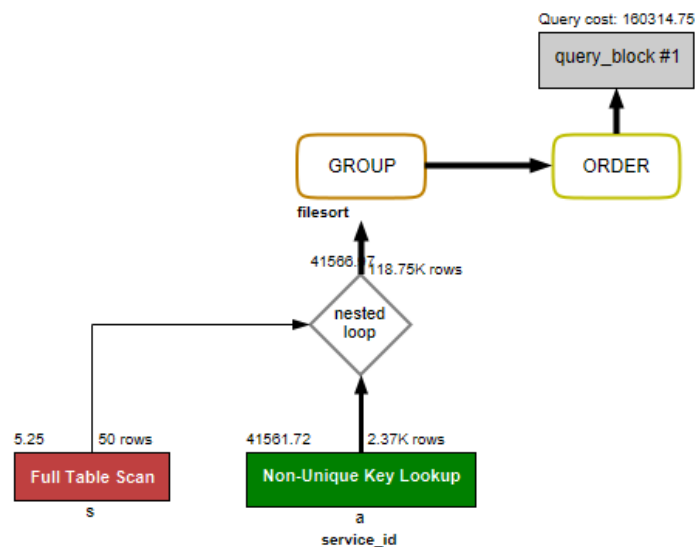


Рис. 16. Графическое представление последовательности выполнения запроса 3

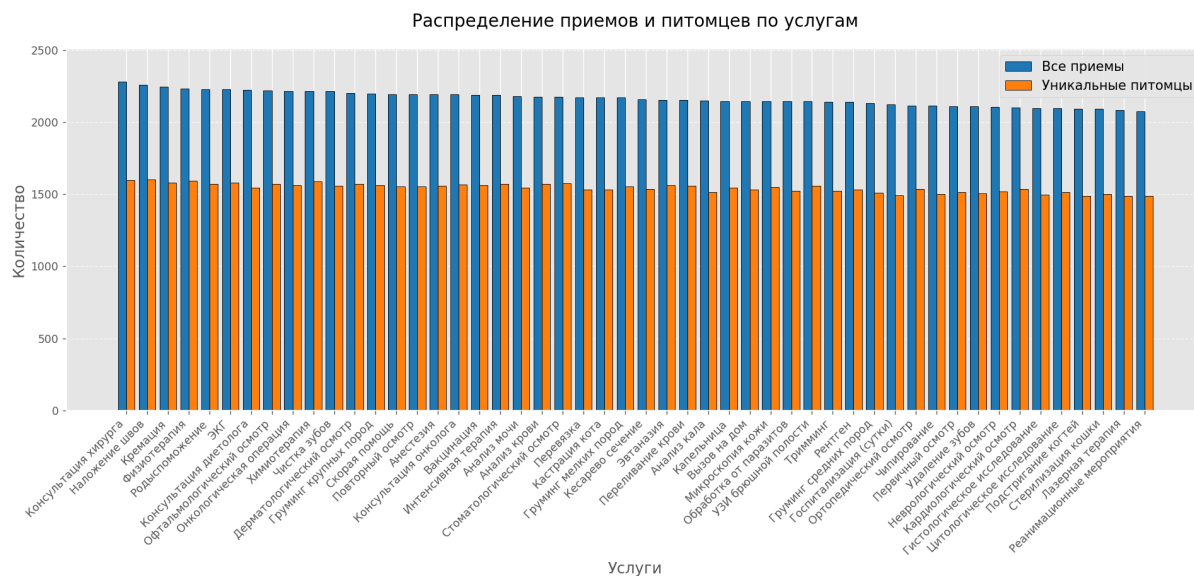


Рис. 17. Гистограмма запроса 3

4.5 Запрос 4.1

Формулировка запроса

Найти товары, которые покупались чаще всего.

Реляционная формула запроса

$$\pi \left(\begin{array}{l} \beta(\text{product.name AS product_name}), \\ \beta(\text{COUNT(purchase.product_id) AS total_purchase}) \end{array} \right)$$

$\text{purchase} \bowtie (\text{purchase.product_id} = \text{product.product_id}) \text{ product}$
 $\gamma(\text{service.service_id, service.name})$

Код запроса

```
1 SELECT
2     product.name AS product_name,
3     COUNT(purchase.product_id) AS total_purchases
4 FROM
5     purchase
6     JOIN
7     product ON purchase.product_id = product.product_id
8 GROUP BY purchase.product_id
9 HAVING COUNT(purchase.product_id) = (SELECT
10     MAX(purchase_count)
11 FROM
12     (SELECT
13         COUNT(product_id) AS purchase_count
14     FROM
15         purchase
16     GROUP BY product_id) AS counts);
```

Результаты запроса

В результате выполнения запроса было выведено 2 строки. Время выполнения запроса составило 0.053 секунды. Результат запроса представлен на рис. 18. Табличное и графическое представления последовательности выполнения запроса представлены на рис. 19 и 20 соответственно.

	product_name	total_purchases
►	Корм сухой хозяйка	39
	Корм консервы степь	39

Рис. 18. Результат выполнения запроса 4.1 в виде таблицы

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	product		ALL	PRIMARY				700	100.00	Using temporary
1	PRIMARY	purchase		ref	product_id	product_id	4	vetclinic.product.product...	16	100.00	Using index
2	SUBQUERY	<derived3>		ALL					11...	100.00	
3	DERIVED	purchase		index	product_id	product_id	4		11...	100.00	Using index

Рис. 19. Табличное представление последовательности выполнения запроса 4.1

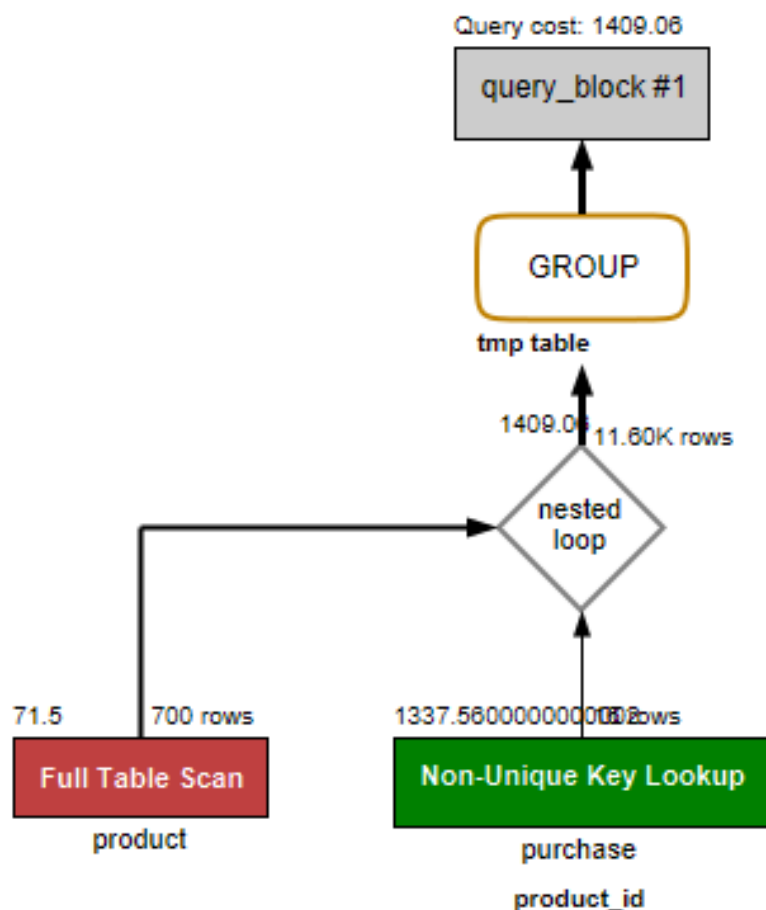


Рис. 20. Графическое представление последовательности выполнения запроса 4.1

4.6 Запрос 4.2

Формулировка запроса

Найти товары, которые покупались реже всего.

Код запроса

```
1 SELECT
2     product.name AS product_name,
3     COUNT(purchase.product_id) AS total_purchases
4 FROM
5     purchase
6     JOIN
7     product ON purchase.product_id = product.product_id
8 GROUP BY purchase.product_id
9 HAVING COUNT(purchase.product_id) = (SELECT
10     MIN(purchase_count)
11 FROM
12     (SELECT
13         COUNT(product_id) AS purchase_count
14     FROM
15         purchase
16     GROUP BY product_id) AS counts);
```

Результаты запроса

В результате выполнения запроса была выведена 1 строка. Время выполнения запроса составило 0.053 секунды. Результат запроса представлен на рис. 21. Табличное и графическое представления последовательности выполнения запроса представлены на рис. 22 и 23 соответственно.



	product_name	total_purchases
►	Лекарства противовоспалительные следова...	1

Рис. 21. Результат выполнения запроса 4.2 в виде таблицы

4.7 Запрос 5

Формулировка запроса

Посчитать число владельцев с одинаковым количеством питомцев.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	product		ALL	PRIMARY				700	100.00	Using temporary
1	PRIMARY	purchase		ref	product_id	product_id	4	vetclinic.product.product_id	16	100.00	Using index
2	SUBQUERY	<derived3>		ALL					11...	100.00	
3	DERIVED	purchase		index	product_id	product_id	4		11...	100.00	Using index

Рис. 22. Табличное представление последовательности выполнения запроса 4.2

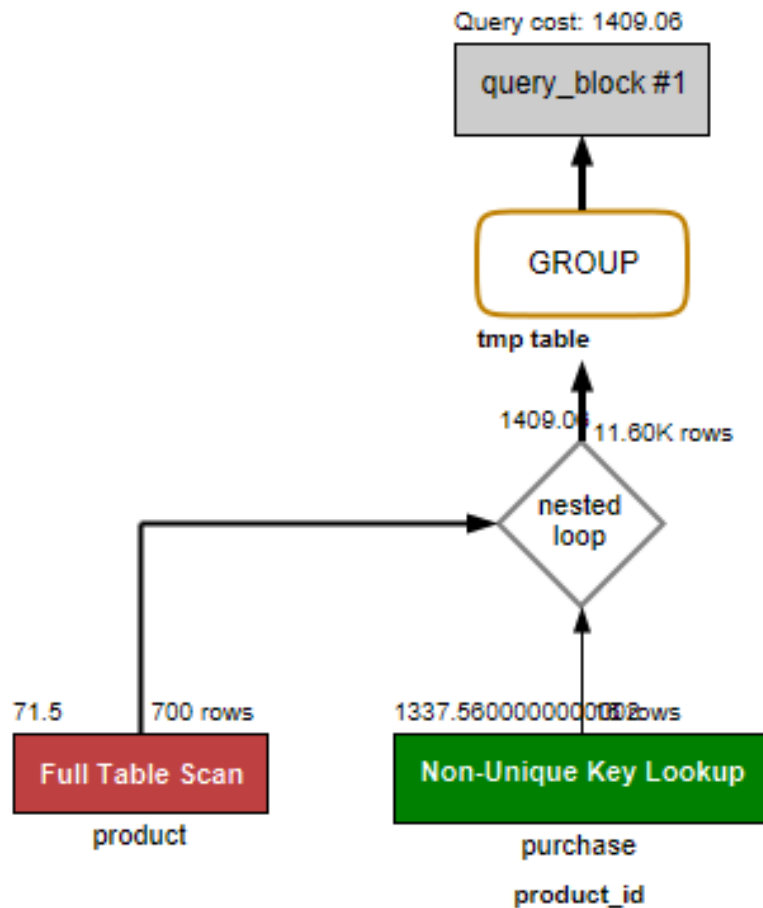


Рис. 23. Графическое представление последовательности выполнения запроса 4.2

Код запроса

```

1 SELECT
2     pet_counts.pet_count,
3     COUNT(pet_counts.owner_id) AS owner_count
4 FROM (
5     SELECT
6         o.owner_id,
7         COUNT(p.pet_id) AS pet_count
8     FROM

```

```

9         owners o
10    LEFT JOIN
11        pet p ON o.owner_id = p.owner_id
12    GROUP BY
13        o.owner_id
14 ) AS pet_counts
15 GROUP BY
16     pet_counts.pet_count
17 ORDER BY
18     pet_counts.pet_count;

```

Текстовое объяснение запроса

В запросе используется подзапрос `pet_counts`, который соединяет таблицы `owners` и `pet` через `LEFT JOIN` по полю `owner_id`, группирует результат соединения по `owner_id` и для каждого владельца считает количество его питомцев `COUNT(p.pet_id)`.

Далее основной запрос обрабатывает результаты подзапроса: он группирует данные по количеству питомцев, для каждой группы подсчитывает количество владельцев. В конце происходит сортировка результатов по возрастанию количества питомцев.

Результаты запроса

В результате выполнения запроса было выведено 30 строк. Время выполнения запроса составило 0.043 секунды. Результат запроса представлен на рис. 24. Табличное и графическое представления последовательности выполнения запроса представлены на рис. 25 и 26 соответственно.

	pet_count	owner_count
▶	1	7
	2	7
	3	6
	4	10
	5	3
	6	5
	7	5
	8	6
	9	10
	10	9

Рис. 24. Результат выполнения запроса 5 в виде фрагмента таблицы

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<de...		ALL					3096	100.00	Using temporary; Using filesort
2	DERIVED	o		index	PRIMARY	PRIMARY	4		200	100.00	Using index
2	DERIVED	p		ref	owner_id	owner_id	4	vetclinic.o.owner_id	15	100.00	Using index

Рис. 25. Табличное представление последовательности выполнения запроса 5

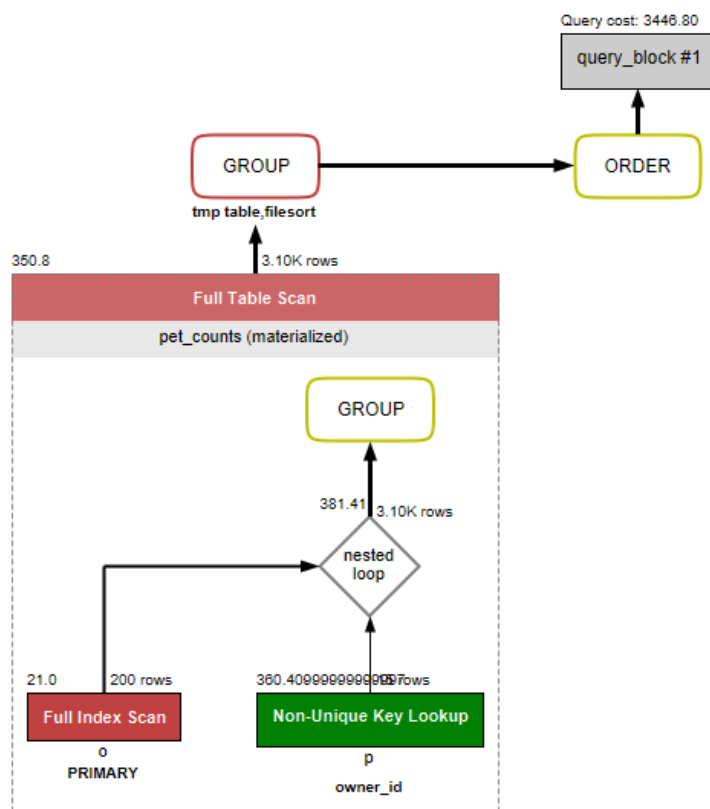


Рис. 26. Графическое представление последовательности выполнения запроса 5

Гистограмма запроса

На рис. 27 представлена 2D-гистограмма, демонстрирующая распределение владельцев по количеству питомцев.

4.8 Запрос 6

Формулировка запроса

Найти администраторов, которые оформили больше покупок, чем администратор А.

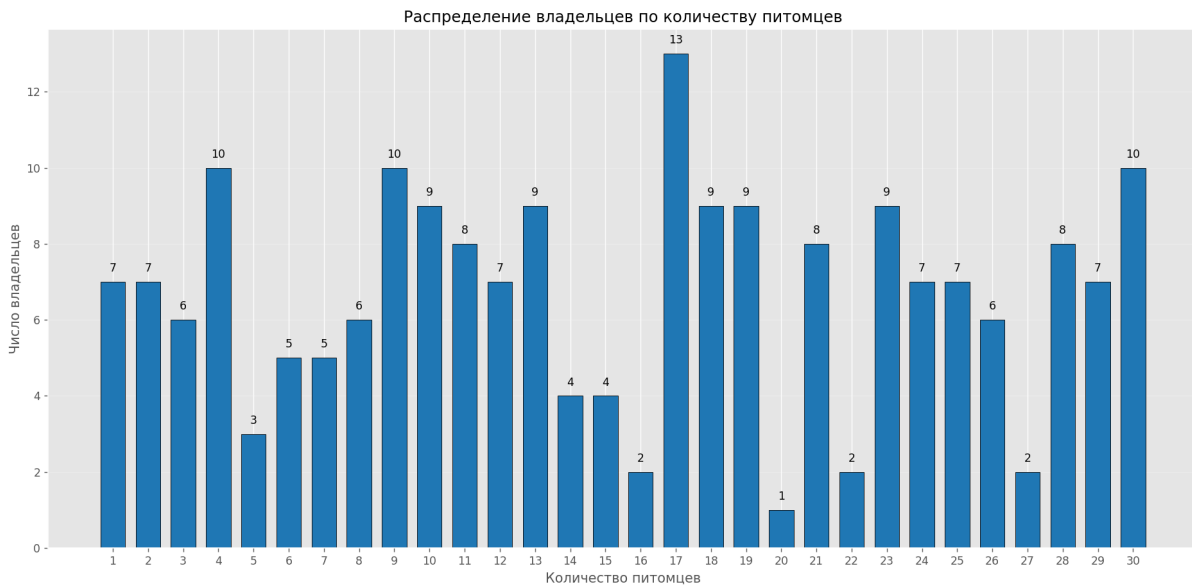


Рис. 27. Гистограмма запроса 5

Код запроса

```

1  SELECT
2      a.admin_id,
3      a.first_name,
4      a.second_name,
5      COUNT(p.purchase_id) AS purchase_count
6  FROM
7      administrator a
8  LEFT JOIN
9      purchase p ON a.admin_id = p.admin_id
10 GROUP BY
11     a.admin_id, a.first_name, a.second_name
12 HAVING
13     COUNT(p.purchase_id) > (
14         SELECT COUNT(p2.purchase_id)
15         FROM administrator a2
16         LEFT JOIN purchase p2 ON a2.admin_id = p2.admin_id
17         WHERE a2.admin_id = 1
18         GROUP BY a2.admin_id
19     )
20 ORDER BY
21     purchase_count DESC;

```

Результаты запроса

В результате выполнения запроса было выведено 7 строк. Время выполнения запроса составило 0.078 секунд. Результат запроса представлен на рис. 28. Табличное и графическое представления последовательности выполнения запроса представлены на рис. 29 и 30 соответственно.

	admin_id	first_name	second_name	purchase_count
►	50	Акулина	Фомина	245
	14	Виктор	Виноградов	243
	22	Харламий	Корнилов	241
	23	Аполлинарий	Козлов	241
	26	Майя	Коновалова	241
	30	Октябрина	Полякова	241
	21	Ювеналий	Попов	240

Рис. 28. Результат выполнения запроса 6 в виде таблицы

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	a		ALL					50	100.00	Using temporary; Using filesort
1	PRIMARY	p		ref	admin_id	admin_id	4	vetclinic.a.admin_id	231	100.00	Using index
2	SUBQUERY	a2		const	PRIMARY	PRIMARY	4	const	1	100.00	Using index
2	SUBQUERY	p2		ref	admin_id	admin_id	4	const	239	100.00	Using index

Рис. 29. Табличное представление последовательности выполнения запроса 6

4.9 Запрос 7

Формулировка запроса

Найти владельцев, которые никогда не обращались к ветеринару А.

Код запроса

```
1 SELECT
2     o.owner_id,
3     o.first_name,
4     o.second_name
5 FROM
6     owners o
7 WHERE
8     o.owner_id NOT IN (
9         SELECT DISTINCT a.owner_id
```

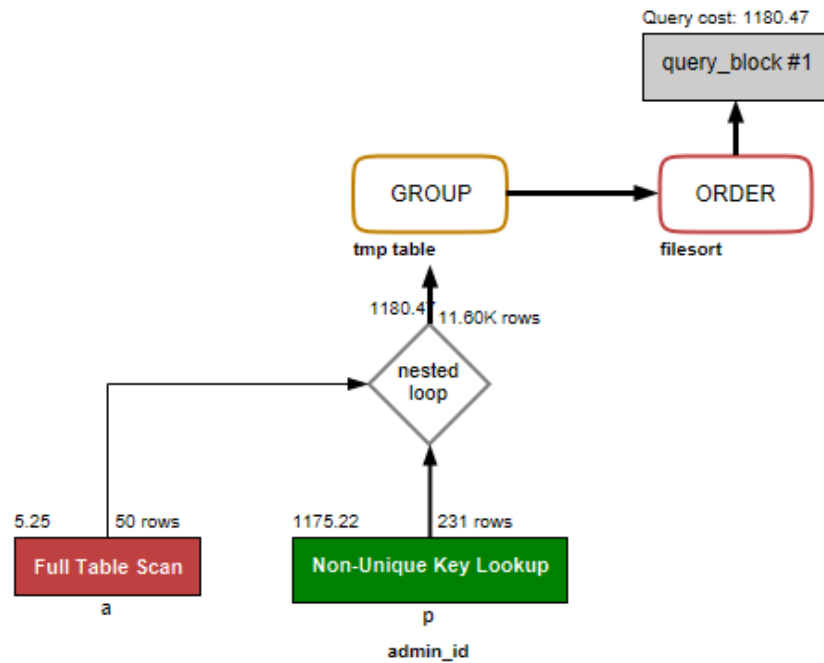


Рис. 30. Графическое представление последовательности выполнения запроса 6

```

10      FROM appointment a
11      JOIN veterinarian_appointment va ON a.appointment_id =
12      ↪ va.appointment_id
12      WHERE va.vet_id = 10
13  );

```

Результаты запроса

В результате выполнения запроса было выведено 4 строки. Время выполнения запроса составило 0.122 секунды. Результат запроса представлен на рис. 31. Табличное и графическое представления последовательности выполнения запроса представлены на рис. 32 и 33 соответственно.

4.10 Запрос 8

Формулировка запроса

Для каждой породы собак и первых 10 ветеринаров посчитать число приемов.

	owner_id	first_name	second_name
►	4	Оксана	Прохорова
	33	Светлана	Блохина
	162	Ратибор	Русаков
	183	Максимильян	Матвеев

Рис. 31. Результат выполнения запроса 7 в виде таблицы

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	o		ALL					200	100.00	
1	SIMPLE	<subquery2>		eq_ref	<auto_distinct_key>	<auto_distinct_key>	5	vetclinic.o.owner_id	1	100.00	Using where; Not exists
2	MATERIALIZED	va		ref	appointment_id,vet_id	vet_id	4	const	3130	100.00	
2	MATERIALIZED	a		eq_ref	PRIMARY,fk_appointment_owner	PRIMARY	4	vetclinic.va.appointment_id	1	100.00	

Рис. 32. Табличное представление последовательности выполнения запроса 7

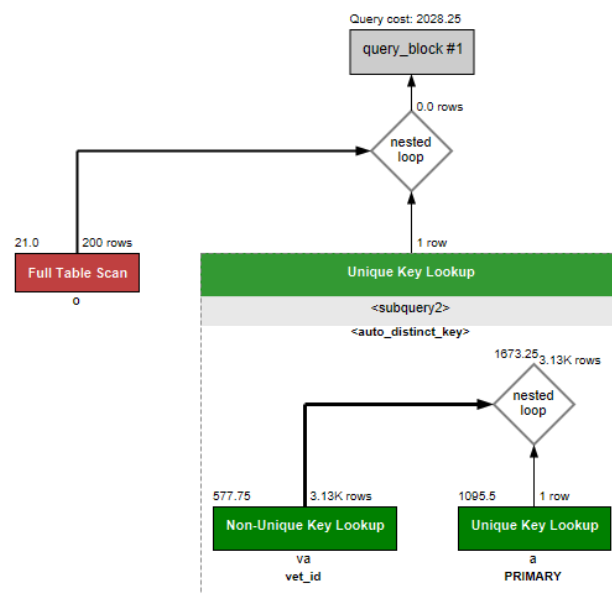


Рис. 33. Графическое представление последовательности выполнения запроса 7

Код запроса

```

1 SELECT
2     b.name AS breed_name,
3     v.vet_id,
4     v.first_name,
5     v.second_name,
6     COUNT(va.vet_app_id) AS appointment_count
7 FROM

```

```

8      (SELECT
9          *
10     FROM
11         veterinarian
12     ORDER BY vet_id
13     LIMIT 10) AS v
14     CROSS JOIN
15     (SELECT
16         b.*
17     FROM
18         breed b
19     JOIN species s ON b.species_id = s.species_id
20     WHERE
21         s.name = 'Собака') AS b
22     LEFT JOIN pet p ON p.breed_id = b.breed_id
23     LEFT JOIN appointment a ON a.pet_id = p.pet_id
24     LEFT JOIN veterinarian_appointment va ON
25         ↪ va.appointment_id = a.appointment_id
26         AND va.vet_id = v.vet_id
27 GROUP BY b.name , v.vet_id , v.first_name , v.second_name
28 ORDER BY b.name , v.vet_id;

```

Результаты запроса

В результате выполнения запроса было выведено 150 строк. Время выполнения запроса составило 0.312 секунд. Результат запроса представлен на рис. 34.

	breed_name	vet_id	first_name	second_name	appointment_count
►	Бигль	1	Полина	Рогова	8
	Бигль	2	Моисей	Кудрявцев	9
	Бигль	3	Леонтий	Орлова	11
	Бигль	4	Гремислав	Кириллов	9
	Бигль	5	Андроник	Федотова	12
	Бигль	6	Гурий	Шестакова	8
	Бигль	7	Серафим	Крюкова	4
	Бигль	8	Мокей	Карпова	6
	Бигль	9	Вера	Афанасьева	10
	Бигль	10	Артений	Соловьева	11

Рис. 34. Результат выполнения запроса 8 в виде фрагмента таблицы

Табличное и графическое представления последовательности выполнения запроса представлены на рис. 35 и 36 соответственно.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	s		ALL	PRIMARY				20	10.00	Using where; Using temporary; Using filesort
1	PRIMARY	b		ref	species_id	species_id	4	vetclinic.s.species_id	13	100.00	
1	PRIMARY	<derived2>		ALL					10	100.00	Using join buffer (hash join)
1	PRIMARY	p		ref	breed_id	breed_id	5	vetclinic.b.breed_id	11	100.00	Using index
1	PRIMARY	a		ref	pet_id	pet_id	4	vetclinic.p.pet_id	34	100.00	Using index
1	PRIMARY	va		ref	appointment_id,vet_id	appointment_id	4	vetclinic.a.appointment_id	1	100.00	Using where
2	DERIVED	veterinarian		index		PRIMARY	4		10	100.00	

Рис. 35. Табличное представление последовательности выполнения запроса 8

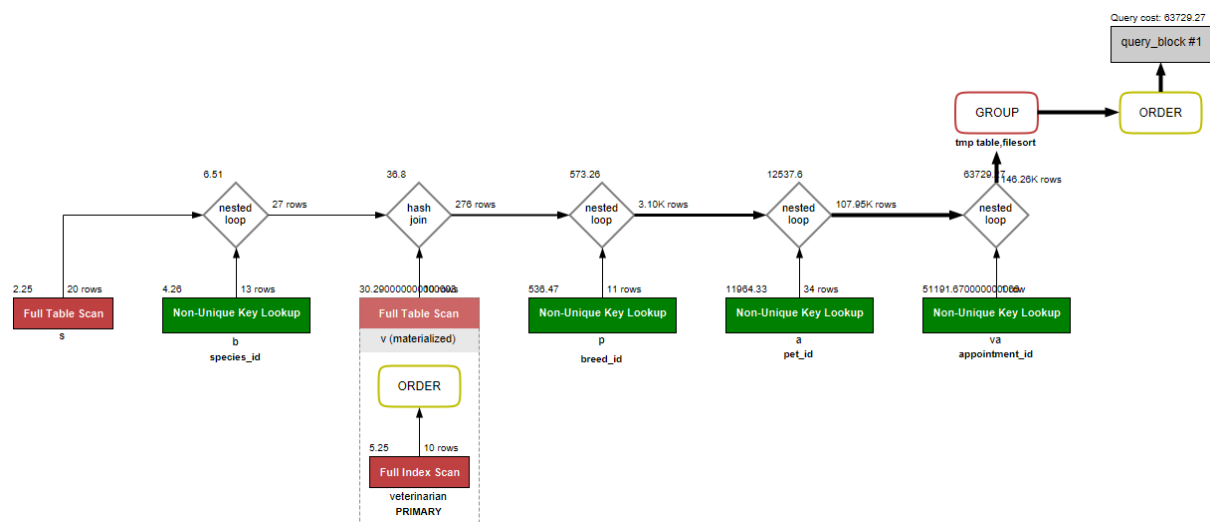


Рис. 36. Графическое представление последовательности выполнения запроса 8

Гистограмма запроса

На рис. 37 представлена 3D-гистограмма, демонстрирующая распределение приемов для каждой породы собаки и первых 10 ветеринаров.

Распределение приемов по породам собак и ветеринарам

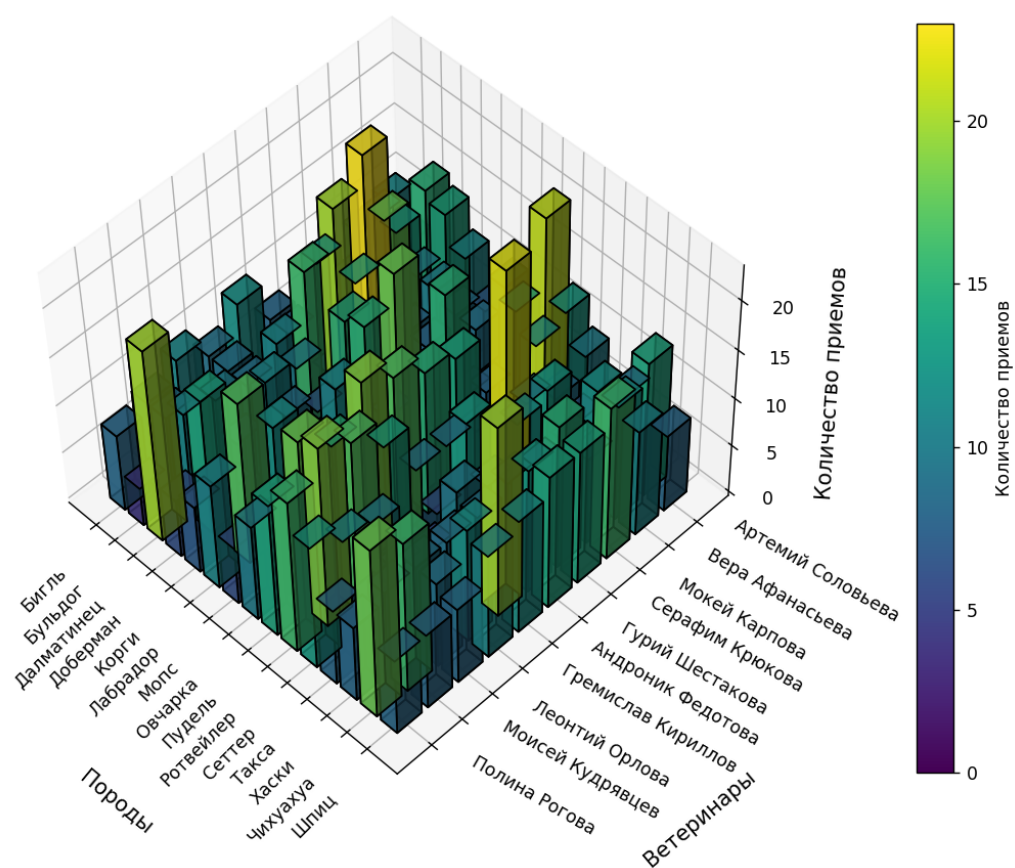


Рис. 37. Гистограмма запроса 8

Заключение

На первом этапе выполнения курсовой работы была выбрана и проанализирована предметная область «Функционирование работы многопрофильной ветеринарной клиники». В ходе описания предметной области было выделено 7 сущностей и 37 атрибутов. На основе описанных процессов и ролей была построена ER-диаграмма, в которой количество атрибутов увеличилось до 42. Также была составлена схема связи объектов, которая содержит в себе 8 сущностей.

На втором этапе была спроектирована схема базы данных, состоящая из 12 таблиц. Для каждой таблицы были перечислены все атрибуты, их типы данных и ограничения. Для схемы базы данных было составлено графическое представление на русском и английском языках. После этого был написан код для создания базы данных и содержащихся в ней таблиц на языке MySQL в командной строке MySQL. Последним шагом база данных была заполнена 280039 записями с помощью программы, написанной на языке Python.

Третьим этапом выполнения работы является написание 8 запросов, сформированных на основе заполненной базы данных. Каждый запрос был написан на языке SQL, после чего был проанализирован с помощью команд EXPLAIN и Visual EXPLAIN.

Курсовая работа была выполнена в MySQL Workbench 8.0 CE, командной строке WINDOWS и IDLE (Python 3.12 64-bit). Работа была выполнена на ноутбуке с процессором 12th Gen Intel(R) Core(TM) i5-1235U 1.30 GHz, с 16 ГБ оперативной памяти и операционной системой Windows 11.

Список использованных источников

- [1] История ветеринарной медицины: краткий курс лекций для студентов 1 курса специальности «Ветеринария» / Сост.: И. Ю. Домницкий // ФГБОУ ВО Саратовский ГАУ. – Саратов, 2016. – 49 с. – [Электронный ресурс] (дата обращения: 23.02.2025).
- [2] ГОСТ Р 55634-2013 Услуги для непродуктивных животных. Общие требования к объектам ветеринарной деятельности. – Введ. 15-10-2013. – М.: Стандартинформ, 2014. – [Электронный ресурс] (дата обращения: 23.02.2025).
- [3] Гражданский кодекс Российской Федерации (часть первая) от 30.11.1994 № 51-ФЗ (ред. от 21.12.2023) // КонсультантПлюс. – [Электронный ресурс] (дата обращения: 23.02.2025).
- [4] PyMySQL Documentation / © Copyright 2023, Inada Naoki and GitHub contributors. – URL: <https://pymysql.readthedocs.io/en/latest/> (дата обращения: 15.04.2025).
- [5] Введение в базы данных, 8-е издание.: Пер. с англ. – М.: Издательский дом «Вильямс», 2005. – 1328 с.: ил. – [Электронный ресурс].

Приложение А «Исходный код программы генерации схемы базы данных»

```
1 CREATE DATABASE vetClinic;
2 USE vetClinic;
3
4 CREATE TABLE species (
5     species_id INT PRIMARY KEY AUTO_INCREMENT,
6     name VARCHAR(14) NOT NULL
7 ) ENGINE=InnoDB;
8
9 CREATE TABLE breed (
10     breed_id INT PRIMARY KEY AUTO_INCREMENT,
11     name VARCHAR(40) NOT NULL,
12     species_id INT NOT NULL,
13     FOREIGN KEY (species_id)
14         REFERENCES species(species_id)
15         ON DELETE CASCADE
16         ON UPDATE CASCADE
17 ) ENGINE=InnoDB;
18
19 CREATE TABLE owners (
20     owner_id INT PRIMARY KEY AUTO_INCREMENT,
21     first_name VARCHAR(50) NOT NULL,
22     second_name VARCHAR(50) NOT NULL,
23     middle_name VARCHAR(50),
24     phone_number VARCHAR(20) NOT NULL,
25     email VARCHAR(254),
26     address VARCHAR(100)
27 ) ENGINE=InnoDB;
28
29 CREATE TABLE pet (
30     pet_id INT PRIMARY KEY AUTO_INCREMENT,
31     nickname VARCHAR(50) NOT NULL,
32     birth_date DATE,
33     sex VARCHAR(7) NOT NULL,
34     allergy VARCHAR(100),
35     chip_number INT,
36     owner_id INT NOT NULL,
37     species_id INT NOT NULL,
38     breed_id INT,
39     FOREIGN KEY (owner_id)
40         REFERENCES owners(owner_id)
41         ON DELETE RESTRICT
42         ON UPDATE CASCADE,
43     FOREIGN KEY (species_id)
44         REFERENCES species(species_id)
45         ON DELETE RESTRICT
46         ON UPDATE CASCADE,
47     FOREIGN KEY (breed_id)
48         REFERENCES breed(breed_id)
49         ON DELETE SET NULL
50         ON UPDATE CASCADE
51 ) ENGINE=InnoDB;
52
53 ALTER TABLE owners
54 ADD COLUMN pet_id INT NOT NULL,
55 ADD FOREIGN KEY (pet_id) REFERENCES pet(pet_id)
56     ON DELETE CASCADE
57     ON UPDATE CASCADE;
58
59 CREATE TABLE veterinarian (
60     vet_id INT PRIMARY KEY AUTO_INCREMENT,
61     first_name VARCHAR(50) NOT NULL,
62     second_name VARCHAR(50) NOT NULL,
63     middle_name VARCHAR(50),
64     phone_number VARCHAR(20) NOT NULL,
65     email VARCHAR(254) NOT NULL,
```

```

66     specialization VARCHAR(15) NOT NULL,
67     qualification VARCHAR(6) NOT NULL
68 ) ENGINE=InnoDB;
69
70 CREATE TABLE service (
71     service_id INT PRIMARY KEY AUTO_INCREMENT,
72     name VARCHAR(30) NOT NULL,
73     price DECIMAL(10,2) NOT NULL
74 ) ENGINE=InnoDB;
75
76 CREATE TABLE administrator (
77     admin_id INT PRIMARY KEY AUTO_INCREMENT,
78     first_name VARCHAR(50) NOT NULL,
79     second_name VARCHAR(50) NOT NULL,
80     middle_name VARCHAR(50),
81     phone_number VARCHAR(20) NOT NULL,
82     email VARCHAR(254) NOT NULL
83 ) ENGINE=InnoDB;
84
85 CREATE TABLE appointment (
86     appointment_id INT PRIMARY KEY AUTO_INCREMENT,
87     status VARCHAR(10) NOT NULL,
88     appointment_date DATE NOT NULL,
89     appointment_time TIME NOT NULL,
90     reason VARCHAR(100) NOT NULL,
91     conclusion VARCHAR(200),
92     treatment VARCHAR(200),
93     service_id INT NOT NULL,
94     pet_id INT NOT NULL,
95     owner_id INT NOT NULL,
96     admin_id INT NOT NULL,
97     FOREIGN KEY (service_id)
98         REFERENCES service(service_id)
99         ON DELETE RESTRICT
100        ON UPDATE CASCADE,
101     FOREIGN KEY (pet_id)
102         REFERENCES pet(pet_id)
103         ON DELETE RESTRICT
104        ON UPDATE CASCADE,
105     FOREIGN KEY (owner_id)
106         REFERENCES owners(owner_id)
107         ON DELETE RESTRICT
108        ON UPDATE CASCADE,
109     FOREIGN KEY (admin_id)
110         REFERENCES administrator(admin_id)
111         ON DELETE RESTRICT
112        ON UPDATE CASCADE
113 ) ENGINE=InnoDB;
114
115 ALTER TABLE appointment
116 ADD COLUMN owner_id INT NOT NULL AFTER pet_id,
117 ADD CONSTRAINT fk_appointment_owner
118     FOREIGN KEY (owner_id)
119     REFERENCES owners(owner_id)
120     ON DELETE RESTRICT
121     ON UPDATE CASCADE;
122
123 CREATE TABLE veterinarian_appointment (
124     vet_app_id INT PRIMARY KEY AUTO_INCREMENT,
125     appointment_id INT NOT NULL,
126     vet_id INT NOT NULL,
127     FOREIGN KEY (appointment_id)
128         REFERENCES appointment(appointment_id)
129         ON DELETE CASCADE
130        ON UPDATE CASCADE,
131     FOREIGN KEY (vet_id)
132         REFERENCES veterinarian(vet_id)
133         ON DELETE RESTRICT
134        ON UPDATE CASCADE
135 ) ENGINE=InnoDB;
136

```

```

137 CREATE TABLE supplier (
138     supplier_id INT PRIMARY KEY AUTO_INCREMENT,
139     name VARCHAR(50) NOT NULL
140 ) ENGINE=InnoDB;
141
142 CREATE TABLE product (
143     product_id INT PRIMARY KEY AUTO_INCREMENT,
144     name VARCHAR(50) NOT NULL,
145     country VARCHAR(15) NOT NULL,
146     price DECIMAL(10,2) NOT NULL,
147     supplier_id INT NOT NULL,
148     FOREIGN KEY (supplier_id)
149         REFERENCES supplier(supplier_id)
150         ON DELETE RESTRICT
151         ON UPDATE CASCADE
152 ) ENGINE=InnoDB;
153
154 CREATE TABLE purchase (
155     purchase_id INT PRIMARY KEY AUTO_INCREMENT,
156     purchase_date DATE NOT NULL,
157     product_id INT NOT NULL,
158     owner_id INT NOT NULL,
159     admin_id INT NOT NULL,
160     FOREIGN KEY (product_id)
161         REFERENCES Product(product_id)
162         ON DELETE RESTRICT
163         ON UPDATE CASCADE,
164     FOREIGN KEY (owner_id)
165         REFERENCES owners(owner_id)
166         ON DELETE RESTRICT
167         ON UPDATE CASCADE,
168     FOREIGN KEY (admin_id)
169         REFERENCES administrator(admin_id)
170         ON DELETE RESTRICT
171         ON UPDATE CASCADE
172 ) ENGINE=InnoDB;

```

Приложение Б «Исходный код программы заполнения базы данных»

Листинг 1. Подключение к базе данных

```
1 db_config = {
2     'user': 'root',
3     'password': 'J15ytbdaY12lh',
4     'host': 'localhost',
5     'database': 'vetclinic'
6 }
```

Листинг 2. Генерация значений для таблиц

```
1 from faker import Faker
2 import random
3
4 fake = Faker('ru_RU')
5
6 def generate_species():
7     species_list = [
8         "Собака", "Кошка", "Попугай", "Хомяк", "Черепаха",
9         "Кролик", "Змея", "Ящерица", "Птица", "Рыбка",
10        "Морская свинка", "Крыса", "Хорёк", "Шиншилла", "Улитка",
11        "Игуана", "Паук", "Хамелеон", "Песчанка", "Пони"
12    ]
13    return [{"name": name} for name in species_list]
14
15 BREEDS_BY_SPECIES = {
16     "Собака": ["Лабрадор", "Такса", "Овчарка", "Бульдог", "Пудель",
17               "Доберман", "Ротвейлер", "Чихуахуа", "Шпиц", "Хаски",
18               "Далматинец", "Бигль", "Мопс", "Корги", "Сеттер"],
19     "Кошка": ["Персидская", "Сиамская", "Британская", "Мейн-кун", "Сфинкс",
20               "Бенгальская", "Русская голубая", "Шотландская", "Ориентал", "Рэгдолл",
21               "Бурма", "Абиссинская", "Норвежская лесная", "Сибирская", "Балинезийская"],
22     "Попугай": ["Волнистый", "Корелла", "Ара", "Какаду", "Жако",
23               "Неразлучник", "Розелла", "Амазон", "Ожереловый", "Лори",
24               "Аратинга", "Какарик", "Сенегальский", "Эклектус", "Квакер"],
25     "Хомяк": ["Джунгарский", "Сирийский", "Кэмпбелла", "Роборовский",
26               "Китайский", "Эдварда", "Радеау", "Брандта", "Соболиный",
27               "Жемчужный", "Коричневый", "Оранжевый", "Белый", "Золотой", "Черный"],
28     "Черепаха": ["Красноухая", "Среднеазиатская", "Болотная", "Зеленая", "Греческая",
29               "Египетская", "Индийская", "Каспийская", "Мексиканская", "Леопардовая",
30               "Китайский трионикс", "Мускусная", "Иловая"],
31     "Кролик": [
32         "Вислоухий баран", "Голландский", "Ангорский", "Рекс", "Гермелин",
33         "Бабочка", "Новозеландский", "Калифорнийский", "Серебристый", "Фландр",
34         "Львиноголовый", "Карликовый", "Огневка", "Бельгийский великан"
35     ],
36     "Змея": [
37         "Королевский питон", "Молочная змея", "Кукурузная змея", "Боа констриктор",
38         "Маисовый полоз", "Тигровый питон", "Радужный удав", "Уж обыкновенный",
39         "Черная мамба", "Зеленая мамба", "Гадюка", "Щитомордник", "Анаконда"
40     ],
41     "Ящерица": [
42         "Геккон", "Игуана", "Агама", "Хамелеон", "Сцинк", "Варан",
43         "Анолис", "Молох", "Поясхвост", "Живородящая", "Плащеносная",
44         "Водяной дракон", "Бородатая агама"
45     ],
46     "Птица": ["Ворон", "Дрозд", "Канарейка", "Грач", "Петух", "Снегирь",
47               "Утка", "Лебедь", "Щегол", "Сова", "Голубь", "Воробей", "Синица",
48               "Сойка", "Дятел"],
49     "Рыбка": [
50         "Гуппи", "Скалярия", "Петушок", "Меченосец", "Моллинезия",
51         "Данио", "Барбус", "Неон", "Золотая рыбка", "Карп кои",
52         "Дискус", "Астронотус", "Пирания", "Арована", "Сом"
```

```

53 ],
54 "Морская свинка": [
55     "Американская", "Абиссинская", "Перуанская", "Тексель", "Шелти",
56     "Коронет", "Рекс", "Тедди", "Альпака", "Мерино", "Лункария",
57     "Белая", "Трехцветная", "Гималайская"
58 ],
59 "Крыса": [
60     "Стандарт", "Дамбо", "Рекс", "Бесхвостая", "Сфинкс", "Сатин",
61     "Пуховая", "Альбинос", "Капюшон", "Беркшир", "Английская",
62     "Русская голубая", "Сиамская"
63 ],
64 "Хорёк": [
65     "Альбинос", "Соболь", "Шампань", "Корица", "Черный",
66     "Белый", "Пастель", "Серебристый", "Темноглазый белый",
67     "Панда", "Блейз", "Далматинец", "Шоколадный"
68 ],
69 "Шиншилла": [
70     "Стандарт серая", "Черный бархат", "Белая", "Сапфир", "Фиолет",
71     "Бежевая", "Гомобежевая", "Эбони", "Коричневая", "Гетеробежевая",
72     "Пастель", "Бело-розовая", "Антрацит"
73 ],
74 "Улитка": [
75     "Ахатина", "Виноградная", "Катушка", "Физа", "Мелания",
76     "Хелена", "Мариза", "Неретина", "Ампулярия", "Теодоксус",
77     "Лужанка", "Цепя", "Битиния"
78 ],
79 "Игуана": [
80     "Зеленая", "Синяя", "Красная", "Пустынная", "Морская",
81     "Чаквелла", "Анолис", "Василиск", "Шлемоносная", "Кольцехвостая",
82     "Шипохвостая", "Шишковатая", "Шлемоносый василиск"
83 ],
84 "Паук": [
85     "Птицеед", "Крестовик", "Скакун", "Волк", "Бродяга",
86     "Черная вдова", "Тарантул", "Сенокосец", "Домовый",
87     "Серебрянка", "Каракурт", "Бразильский странствующий",
88     "Мышинный"
89 ],
90 "Хамелеон": [
91     "Йеменский", "Пантеровый", "Джексона", "Ковровый", "Четырехрогий",
92     "Малый", "Гигантский", "Фишера", "Намаква", "Брукезия",
93     "Рудис", "Хохлатый", "Карликовый"
94 ],
95 "Песчанка": [
96     "Монгольская", "Когтистая", "Индийская", "Персидская", "Сундевалла",
97     "Толстохвостая", "Карликовая", "Жирнохвостая", "Короткоухая",
98     "Пушистохвостая", "Белуджистанская", "Египетская", "Королевская"
99 ],
100 "Пони": ["Карликовый", "Золотой", "Большой", "Волнистый", "Белый",
101           "В яблоко", "Серый", "Черный", "Голубой", "Пятнистый",
102           "Полосатый", "Пушистый", "Китайский", "Японский", "Коричневый"]
103 }
104
105 def generate_breeds(species_list):
106     breeds = []
107     for species in species_list:
108         species_name = species["name"]
109         species_id = species["species_id"]
110         for breed_name in random.sample(BREEDS_BY_SPECIES.get(species_name, []),
111                                         min(15, len(BREEDS_BY_SPECIES.get(species_name, []))
112                                         )):
113             breeds.append({
114                 "name": breed_name,
115                 "species_id": species_id
116             })
117     return breeds
118
119 def generate_suppliers(count=50):
120     suppliers = []
121     for _ in range(count):
122         suppliers.append({
123             'name': fake.company()

```

```

123     })
124     return suppliers
125
126 def generate_veterinarians(count=50):
127     veterinarians = []
128     specializations = [
129         'Хирург', 'Терапевт', 'Офтальмолог', 'Дерматолог',
130         'Кардиолог', 'Невролог', 'Онколог', 'Стоматолог',
131         'Уролог', 'Лаборант'
132     ]
133     qualifications = ['Высшая', 'Первая', 'Вторая']
134
135     for _ in range(count):
136         veterinarians.append({
137             'first_name': fake.first_name_male() if random.choice([True, False]) else
138                 fake.first_name_female(),
139             'second_name': fake.last_name_male() if random.choice([True, False]) else
140                 fake.last_name_female(),
141             'middle_name': fake.middle_name_male() if random.choice([True, False]) else
142                 fake.middle_name_female(),
143             'phone_number': fake.phone_number(),
144             'email': fake.email(),
145             'specialization': random.choice(specializations),
146             'qualification': random.choice(qualifications)
147         })
148     return veterinarians
149
150 def generate_administrators(count=50):
151     administrators = []
152     for _ in range(count):
153         gender = random.choice(['male', 'female'])
154         administrators.append({
155             'first_name': fake.first_name_male() if gender == 'male' else fake.first_name_
156                 _female(),
157             'second_name': fake.last_name_male() if gender == 'male' else fake.last_name_
158                 female(),
159             'middle_name': fake.middle_name_male() if gender == 'male' else fake.middle_
160                 name_female(),
161             'phone_number': f"+7({fake.random_int(900, 999)}){fake.random_int(100, 999)}-
162                 {fake.random_int(10, 99)}-{fake.random_int(10, 99)}",
163             'email': f"{fake.user_name()}@{fake.random_int(1, 99)}{fake.domain_name()}"
164         })
165     return administrators
166
167 def generate_services():
168     services = [
169         {"name": "Первичный осмотр", "price": 1500.00},
170         {"name": "Повторный осмотр", "price": 1000.00},
171         {"name": "Вакцинация", "price": 2000.00},
172         {"name": "Чипирование", "price": 2500.00},
173         {"name": "Кастрация кота", "price": 5000.00},
174         {"name": "Стерилизация кошки", "price": 8000.00},
175         {"name": "УЗИ брюшной полости", "price": 3500.00},
176         {"name": "Рентген", "price": 3000.00},
177         {"name": "Анализ крови", "price": 2500.00},
178         {"name": "Чистка зубов", "price": 4000.00},
179         {"name": "Удаление зубов", "price": 6000.00},
180         {"name": "Обработка от паразитов", "price": 1800.00},
181         {"name": "Подстригание когтей", "price": 800.00},
182         {"name": "Перевязка", "price": 1200.00},
183         {"name": "Наложение швов", "price": 3500.00},
184         {"name": "Капельница", "price": 2000.00},
185         {"name": "Госпитализация (сутки)", "price": 4500.00},
186         {"name": "ЭКГ", "price": 2800.00},
187         {"name": "Анализ мочи", "price": 1500.00},
188         {"name": "Анализ кала", "price": 1700.00},
189         {"name": "Тримминг", "price": 2500.00},
190         {"name": "Груминг мелких пород", "price": 3500.00},
191         {"name": "Груминг средних пород", "price": 4500.00},
192         {"name": "Груминг крупных пород", "price": 6000.00},

```

```

187     {"name": "Вызов на дом", "price": 3000.00},
188     {"name": "Эвтаназия", "price": 4000.00},
189     {"name": "Кремация", "price": 7000.00},
190     {"name": "Скорая помощь", "price": 5000.00},
191     {"name": "Консультация диетолога", "price": 2500.00},
192     {"name": "Консультация хирурга", "price": 3000.00},
193     {"name": "Консультация онколога", "price": 3500.00},
194     {"name": "Физиотерапия", "price": 2800.00},
195     {"name": "Лазерная терапия", "price": 3200.00},
196     {"name": "Химиотерапия", "price": 8500.00},
197     {"name": "Онкологическая операция", "price": 15000.00},
198     {"name": "Офтальмологический осмотр", "price": 2200.00},
199     {"name": "Стоматологический осмотр", "price": 1800.00},
200     {"name": "Дерматологический осмотр", "price": 2000.00},
201     {"name": "Кардиологический осмотр", "price": 3500.00},
202     {"name": "Неврологический осмотр", "price": 3800.00},
203     {"name": "Ортопедический осмотр", "price": 3000.00},
204     {"name": "Родыспоможение", "price": 10000.00},
205     {"name": "Кесарево сечение", "price": 12000.00},
206     {"name": "Микроскопия кожи", "price": 1800.00},
207     {"name": "Цитологическое исследование", "price": 2500.00},
208     {"name": "Гистологическое исследование", "price": 5000.00},
209     {"name": "Анестезия", "price": 3000.00},
210     {"name": "Реанимационные мероприятия", "price": 6000.00},
211     {"name": "Интенсивная терапия", "price": 4500.00},
212     {"name": "Переливание крови", "price": 8000.00}
213 ]
214 return services
215
216 def generate_products(supplier_ids, count=700):
217     products = []
218     categories = {
219         'Корм': ['сухой', 'влажный', 'консервы', 'премиум', 'холистик'],
220         'Лекарства': ['антибиотики', 'витамины', 'противопаразитарные', 'анальгетики', 'п
                ротивовоспалительные'],
221         'Аксессуары': ['ошейники', 'поводки', 'миски', 'лежанки', 'переноски'],
222         'Гигиена': ['шампуни', 'расчески', 'зубные щетки', 'пеленки', 'салфетки'],
223         'Игрушки': ['мячи', 'пищалки', 'интерактивные', 'канаты', 'грызунки']
224     }
225
226     countries = ['Россия', 'Германия', 'Франция', 'США', 'Великобритания', 'Китай', 'Итал
                ия', 'Испания']
227
228     supplier_cycle = supplier_ids * (count // len(supplier_ids))
229     remaining = count % len(supplier_ids)
230     supplier_cycle += supplier_ids[:remaining]
231
232     for _ in range(count):
233         category = random.choice(list(categories.keys()))
234         subcategory = random.choice(categories[category])
235         products.append({
236             'name': f"{category} {subcategory} {fake.word()}",
237             'country': random.choice(countries),
238             'price': round(random.uniform(50, 5000), 2),
239             'supplier_id': random.choice(supplier_ids)
240         })
241     return products
242
243 def generate_owners(count=200):
244     owners = []
245     for _ in range(count):
246         gender = random.choice(['male', 'female'])
247         owners.append({
248             'first_name': fake.first_name_male() if gender == 'male' else fake.first_name_
                _female(),
249             'second_name': fake.last_name_male() if gender == 'male' else fake.last_name_
                _female(),
250             'middle_name': fake.middle_name_male() if gender == 'male' else fake.middle_
                _name_female(),
251             'phone_number': f"+7({fake.random_int(900, 999)}){fake.random_int(100, 999)
                }-{fake.random_int(10, 99)}-{fake.random_int(10, 99)}",

```

```

252         'email': f"{fake.user_name()}{random.randint(1, 99)}@{fake.domain_name()}",
253         'address': f"{fake.city()}, ул. {fake.street_name()}, д. {fake.building_
                number()}"
254     })
255     return owners
256
257 def generate_pets(owner_ids, species_list, breed_list, pets_per_owner=(1, 30)):
258     pets = []
259     pet_names = {
260         'Собака': ['Шарик', 'Рекс', 'Джек', 'Белка', 'Тузик', 'Лорд', 'Цезарь',
261                  'Бобик', 'Ральф', 'Арчи', 'Граф', 'Дейзи', 'Лайма', 'Зевс', 'Оскар', 'Ч
                арли', 'Тедди'],
262         'Кошка': ['Мурзик', 'Васька', 'Пушистик', 'Снежок', 'Рыжик', 'Персик', 'Барсик',
263                  'Мурка',
264                  'Симба', 'Луна', 'Оливер', 'Лео', 'Мия', 'Тигра', 'Сэм', 'Люси', 'Гарфил
                д', 'Нюша'],
265         'Попугай': ['Кеша', 'Гоша', 'Рио', 'Арчи', 'Яша', 'Кира', 'Лора', 'Чижик',
266                    'Коко', 'Жак', 'Пип', 'Рики', 'Чико', 'Бонни', 'Клео', 'Лори'],
267         'Хомяк': ['Пушок', 'Нюша', 'Боня', 'Фунтик', 'Пиксель', 'Кнопа', 'Сеня', 'Зефирка
                ',
268                  'Хома', 'Буся', 'Мотя', 'Кругляш', 'Цуки', 'Филя', 'Шуша', 'Плюша'],
269         'Черепаха': ['Тоша', 'Шелл', 'Донни', 'Лео', 'Рафа', 'Мики', 'Сплинтер', 'Вальт',
270                     'Спиди', 'Касси', 'Тортилла', 'Бабблз', 'Шелдон', 'Квази', 'Танк'],
271         'Кролик': ['Роджер', 'Багз', 'Лола', 'Снежок', 'Пухля', 'Флаффи', 'Банни', 'Джасп
                ер',
272                  'Орео', 'Коко', 'Питер', 'Дейзи', 'Макс', 'Лулу', 'Симба'],
273         'Змея': ['Нага', 'Каа', 'Зигги', 'Сайрен', 'Пайтон', 'Медяна', 'Слизи', 'Вайпер',
274                 'Барон', 'Гиза', 'Спаркл', 'Эсмеральда', 'Оникс', 'Салазар'],
275         'Ящерица': ['Годзилла', 'Иго', 'Рекс', 'Спайк', 'Дарт', 'Скала', 'Зилла', 'Гекко
                ',
276                    'Риптор', 'Драко', 'Твигги', 'Йоши', 'Комодо', 'Сцинк'],
277         'Птица': ['Квизи', 'Твити', 'Блю', 'Скай', 'Санни', 'Пип', 'Чип', 'Дейзи',
278                  'Рио', 'Альба', 'Перси', 'Спарки', 'Феникс', 'Валькирия'],
279         'Рыбка': ['Немо', 'Дори', 'Баббл', 'Голди', 'Флиппер', 'Спот', 'Близзард', 'Неон
                ',
280                  'Азур', 'Коралл', 'Плавник', 'Жабр', 'Скаляр', 'Гуппи'],
281         'Морская свинка': ['Пигги', 'Гвинни', 'Флафф', 'Сквик', 'Бекон', 'Панкейк', 'Мох
                ', 'Пух',
282                  'Шоколад', 'Карамель', 'Пончик', 'Бисквит', 'Пельмень', 'Вафля
                '],
283         'Крыса': ['Рэтчет', 'Рэтту', 'Сквиджи', 'Твич', 'Сникерс', 'Пипсквик', 'Гном', 'Ш
                ашлык',
284                  'Рокки', 'Скраффи', 'Чиззи', 'Рэттиган', 'Сплинтер', 'Тайл'],
285         'Хорёк': ['Фредди', 'Ферби', 'Ласка', 'Джинкс', 'Зип', 'Слим', 'Виззи', 'Фуззи',
286                  'Нудл', 'Сквирт', 'Фигаро', 'Локи', 'Пикси', 'Твинкл'],
287         'Шиншилла': ['Чилла', 'Душка', 'Плюш', 'Шани', 'Чоко', 'Латте', 'Марш', 'Зефир',
288                     'Бамбино', 'Чиби', 'Пуфик', 'Сноу', 'Мисти', 'Эльф'],
289         'Улитка': ['Турбо', 'Шелл', 'Спиди', 'Слизз', 'Баббл', 'Гэри', 'Эскарго', 'Сникер
                ',
290                  'Крулл', 'Спираль', 'Койл', 'Слимп', 'Густо', 'Бабблз'],
291         'Игуана': ['Изи', 'Годзи', 'Спайк', 'Зилла', 'Рекс', 'Драгон', 'Торч', 'Смауг',
292                  'Риптор', 'Скала', 'Огненный', 'Альберт', 'Кома', 'Зорро'],
293         'Паук': ['Питер', 'Шелоб', 'Арахна', 'Веном', 'Тара', 'Фанг', 'Чарли', 'Скай',
294                 'Октобер', 'Инки', 'Спайдер', 'Уиджи', 'Локи', 'Зорг'],
295         'Хамелеон': ['Камео', 'Рэйнбоу', 'Зигги', 'Лео', 'Джаспер', 'Рипли', 'Пантер', 'М
                истик',
296                  'Сэм', 'Блендер', 'Краска', 'Халк', 'Ирис', 'Клевер'],
297         'Песчанка': ['Джерри', 'Санд', 'Джинжер', 'Ниббл', 'Скутер', 'Пип', 'Твич', 'Джам
                пер',
298                  'Бисквит', 'Флик', 'Зип', 'Спарки', 'Флип', 'Бамбл'],
299         'Пони': ['Пинки', 'Твай', 'Рарити', 'Эпл', 'Флаттер', 'Спайк', 'Скут', 'Каденс',
300                 'Санни', 'Луна', 'Стар', 'Шугар', 'Баттер', 'Блу']
301     }
302     used_chip_numbers = set()
303     owner_pet_counts = {owner_id: random.randint(*pets_per_owner) for owner_id in owner_
                ids}
304
305     for owner_id, pet_count in owner_pet_counts.items():
306         for _ in range(pet_count):
307             species = random.choice(species_list)

```

```

308     species_id = species['species_id']
309     species_name = species['name']
310
311     available_breeds = [b for b in breed_list if b['species_id'] == species_id]
312     breed = random.choice(available_breeds) if available_breeds else None
313
314     chip = None
315     if random.random() > 0.7:
316         while True:
317             new_chip = random.randint(100000000, 2147483647)
318             if new_chip not in used_chip_numbers:
319                 used_chip_numbers.add(new_chip)
320                 chip = new_chip
321                 break
322
323     name_options = pet_names.get(species_name, ['Дружок', 'Счастливчик', 'Лапуля'])
324
325     pets.append({
326         'nickname': random.choice(name_options),
327         'birth_date': fake.date_between(start_date='-15y', end_date='-1y').
328             strftime('%Y-%m-%d'),
329         'sex': random.choice(['Male', 'Female']),
330         'allergy': random.choice([None, 'Пыльца', 'Курица', 'Злаки', 'Блошинный ук
331             yc']),
332         'chip_number': chip,
333         'owner_id': owner_id,
334         'species_id': species_id,
335         'breed_id': breed['breed_id'] if breed else None })
336
337     return pets
338
339 def generate_purchases(owner_ids, product_ids, admin_ids, purchases_per_owner=(40, 80)):
340     purchases = []
341     owner_purchase_counts = {owner_id: random.randint(*purchases_per_owner) for owner_id
342         in owner_ids}
343     total_purchases = sum(owner_purchase_counts.values())
344
345     admin_pool = []
346
347     base_per_admin = max(1, total_purchases // len(admin_ids) - 10)
348     for admin_id in admin_ids:
349         admin_pool.extend([admin_id] * base_per_admin)
350
351     remaining = total_purchases - len(admin_pool)
352     if remaining > 0:
353         extra_admins = random.choices(admin_ids, k=remaining)
354         admin_pool.extend(extra_admins)
355     random.shuffle(admin_pool)
356
357     product_weights = [random.uniform(0.3, 1.7) for _ in product_ids] # Разная популярно
358     сть
359     product_pool = random.choices(
360         product_ids,
361         weights=product_weights,
362         k=total_purchases
363     )
364     purchase_index = 0
365     for owner_id, purchase_count in owner_purchase_counts.items():
366         for _ in range(purchase_count):
367             purchases.append({
368                 'purchase_date': fake.date_between(start_date='-2y', end_date='today').
369                     strftime('%Y-%m-%d'),
370                 'product_id': product_pool[purchase_index],
371                 'owner_id': owner_id,
372                 'admin_id': admin_pool[purchase_index]
373             })
374             purchase_index += 1
375
376     return purchases
377

```

```

373 def generate_appointments(pet_list, service_ids, admin_ids, appointments_per_pet=(20, 50)
    ):
374     appointments = []
375     reasons = [
376         "Плановый осмотр", "Вакцинация", "Травма",
377         "Кожные проблемы", "Проблемы с ЖКТ", "Стоматология",
378         "Аллергия", "Кастрация/стерилизация", "Чипирование"
379     ]
380     treatments = [
381         "Назначены лекарства", "Рекомендован отдых",
382         "Проведена операция", "Назначена диета",
383         "Рекомендованы дополнительные анализы", "Проведена обработка"
384     ]
385     conclusions = [
386         "Здоров", "Требуется наблюдение", "Хроническое заболевание",
387         "Острое состояние", "Послеоперационный уход", "Ремиссия"
388     ]
389     pet_appointment_counts = {pet['pet_id']: random.randint(*appointments_per_pet) for
        pet in pet_list}
390     total_appointments = sum(pet_appointment_counts.values())
391
392     admin_appointment_counts = {}
393     remaining = total_appointments
394     num_admins = len(admin_ids)
395
396     for i, admin_id in enumerate(admin_ids[:-1]):
397         min_a = max(20, remaining - 50 * (num_admins - i - 1))
398         max_a = min(50, remaining - 20 * (num_admins - i - 1))
399         if min_a > max_a:
400             count = (min_a + max_a) // 2
401         else:
402             count = random.randint(min_a, max_a)
403
404         admin_appointment_counts[admin_id] = count
405         remaining -= count
406
407     admin_appointment_counts[admin_ids[-1]] = remaining
408
409     admin_pool = []
410     for admin_id, count in admin_appointment_counts.items():
411         admin_pool.extend([admin_id] * count)
412     random.shuffle(admin_pool)
413
414     appointment_index = 0
415     for pet in pet_list:
416         pet_id = pet['pet_id']
417         owner_id = pet['owner_id']
418         pet_services = [random.choice(service_ids) for _ in range(pet_appointment_counts[
            pet_id])]
419         for service_id in pet_services:
420             appointment_date = fake.date_between(start_date='-2y', end_date='today')
421             appointments.append({
422                 'status': random.choice(['Completed', 'Cancelled', 'No-show']),
423                 'appointment_date': appointment_date.strftime('%Y-%m-%d'),
424                 'appointment_time': fake.time(pattern='%H:%M'),
425                 'reason': random.choice(reasons),
426                 'conclusion': random.choice(conclusions) if random.random() > 0.2 else
                    None,
427                 'treatment': random.choice(treatments) if random.random() > 0.3 else None
                    ,
428                 'service_id': service_id,
429                 'pet_id': pet_id,
430                 'owner_id': owner_id,
431                 'admin_id': admin_pool[appointment_index]
432             })
433             appointment_index += 1
434     return appointments
435
436 def generate_vet_appointments(appointment_ids, vet_ids):
437     vet_appointments = []
438     vet_cycle = (vet_ids * ((len(appointment_ids) // len(vet_ids)) + 1))[:len(appointment

```

```

    _ids)]
439 for i, appointment_id in enumerate(appointment_ids):
440     vet_appointments.append({
441         'appointment_id': appointment_id,
442         'vet_id': vet_cycle[i]
443     })
444     if random.random() < 0.3:
445         additional_vets = random.sample(vet_ids, random.randint(1, 2))
446         for vet_id in additional_vets:
447             if vet_id != vet_cycle[i]:
448                 vet_appointments.append({
449                     'appointment_id': appointment_id,
450                     'vet_id': vet_id
451                 })
452 return vet_appointments

```

Листинг 3. Заполнение таблицы Species

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_species
4
5 def fill_species():
6     species = generate_species()
7     conn = mysql.connector.connect(**db_config)
8     cursor = conn.cursor()
9
10    for item in species:
11        cursor.execute(
12            "INSERT INTO species (name) VALUES (%s)",
13            (item["name"],)
14        )
15
16    conn.commit()
17    cursor.close()
18    conn.close()
19    print(f"Добавлено {len(species)} записей в таблицу species!")
20
21 if __name__ == "__main__":
22     fill_species()

```

Листинг 4. Заполнение таблицы Breed

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_species
4 from fake_data import generate_breeds
5
6 def get_existing_species(conn):
7     cursor = conn.cursor()
8     cursor.execute("SELECT species_id, name FROM species")
9     species_list = [{"species_id": row[0], "name": row[1]} for row in cursor.fetchall()]
10    cursor.close()
11    return species_list
12
13 def fill_breed():
14     conn = mysql.connector.connect(**db_config)
15
16     species_list = get_existing_species(conn)
17
18     breeds = generate_breeds(species_list)
19
20     cursor = conn.cursor()
21     for breed in breeds:
22         cursor.execute(
23             "INSERT INTO breed (name, species_id) VALUES (%s, %s)",
24             (breed["name"], breed["species_id"])
25         )
26

```

```

27     conn.commit()
28     cursor.close()
29     conn.close()
30     print(f"Добавлено {len(breeds)} записей в таблицу breed!")
31
32 if __name__ == "__main__":
33     fill_breed()

```

Листинг 5. Заполнение таблицы Owners

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_owners
4
5 def fill_owners():
6     conn = mysql.connector.connect(**db_config)
7     cursor = conn.cursor()
8
9     cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
10    cursor.execute("TRUNCATE TABLE owners")
11    cursor.execute("ALTER TABLE owners AUTO_INCREMENT = 1")
12    cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
13
14    owners = generate_owners(200)
15    for owner in owners:
16        cursor.execute(
17            """INSERT INTO owners
18               (first_name, second_name, middle_name, phone_number, email, address)
19               VALUES (%s, %s, %s, %s, %s, %s)"""
20            , (owner['first_name'], owner['second_name'], owner['middle_name'],
21              owner['phone_number'], owner['email'], owner['address'])
22        )
23
24    conn.commit()
25    cursor.close()
26    conn.close()
27    print(f"Добавлено {len(owners)} владельцев!")
28
29 if __name__ == "__main__":
30     fill_owners()

```

Листинг 6. Заполнение таблицы Pet

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_pets
4
5 def get_owner_ids(conn):
6     cursor = conn.cursor(dictionary=True)
7     cursor.execute("SELECT owner_id FROM owners")
8     return [row['owner_id'] for row in cursor.fetchall()]
9
10 def get_species_list(conn):
11     cursor = conn.cursor(dictionary=True)
12     cursor.execute("SELECT species_id, name FROM species")
13     return cursor.fetchall()
14
15 def get_breed_list(conn):
16     cursor = conn.cursor(dictionary=True)
17     cursor.execute("SELECT breed_id, species_id FROM breed")
18     return cursor.fetchall()
19
20 def fill_pets():
21     conn = mysql.connector.connect(**db_config)
22     cursor = conn.cursor()
23
24     try:
25         cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
26         cursor.execute("TRUNCATE TABLE pet")

```

```

27     cursor.execute("ALTER TABLE pet AUTO_INCREMENT = 1")
28     cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
29
30     owner_ids = get_owner_ids(conn)
31     species_list = get_species_list(conn)
32     breed_list = get_breed_list(conn)
33
34     pets = generate_pets(owner_ids, species_list, breed_list)
35
36     for pet in pets:
37         cursor.execute(
38             """INSERT INTO pet
39                 (nickname, birth_date, sex, allergy, chip_number, owner_id, species_id,
40                  breed_id)
41                 VALUES (%s, %s, %s, %s, %s, %s, %s, %s)""" ,
42             (pet['nickname'], pet['birth_date'], pet['sex'], pet['allergy'],
43              pet['chip_number'], pet['owner_id'], pet['species_id'], pet['breed_id'])
44         )
45     conn.commit()
46     print(f"Добавлено {len(pets)} питомцев!")
47
48 if __name__ == "__main__":
49     fill_pets()

```

Листинг 7. Заполнение таблицы Service

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_services
4
5 def fill_services():
6     conn = mysql.connector.connect(**db_config)
7     cursor = conn.cursor()
8
9     cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
10    cursor.execute("TRUNCATE TABLE service")
11    cursor.execute("ALTER TABLE service AUTO_INCREMENT = 1")
12    cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
13
14    services = generate_services()
15    for service in services:
16        cursor.execute(
17            """INSERT INTO service
18                (name, price)
19                VALUES (%s, %s)""" ,
20            (service['name'], service['price'])
21        )
22
23    conn.commit()
24    cursor.close()
25    conn.close()
26    print(f"Добавлено {len(services)} услуг!")
27
28 if __name__ == "__main__":
29     fill_services()

```

Листинг 8. Заполнение таблицы Administrator

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_administrators
4
5 def fill_administrators():
6     conn = mysql.connector.connect(**db_config)
7     cursor = conn.cursor()
8
9     cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
10    cursor.execute("TRUNCATE TABLE administrator")

```

```

11     cursor.execute("ALTER TABLE administrator AUTO_INCREMENT = 1")
12     cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
13
14     admins = generate_administrators(50)
15     for admin in admins:
16         cursor.execute(
17             """INSERT INTO administrator
18                (first_name, second_name, middle_name, phone_number, email)
19                VALUES (%s, %s, %s, %s, %s)"""",
20             (admin['first_name'], admin['second_name'], admin['middle_name'],
21              admin['phone_number'], admin['email']))
22         )
23
24     conn.commit()
25     cursor.close()
26     conn.close()
27     print(f"Добавлено {len(admins)} администраторов!")
28
29 if __name__ == "__main__":
30     fill_administrators()

```

Листинг 9. Заполнение таблицы Appointment

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_appointments
4
5 def get_pet_list(conn):
6     cursor = conn.cursor(dictionary=True)
7     cursor.execute("SELECT pet_id, owner_id FROM pet")
8     return cursor.fetchall()
9
10 def get_service_ids(conn):
11     cursor = conn.cursor()
12     cursor.execute("SELECT service_id FROM service")
13     return [row[0] for row in cursor.fetchall()]
14
15 def get_admin_ids(conn):
16     cursor = conn.cursor()
17     cursor.execute("SELECT admin_id FROM administrator")
18     return [row[0] for row in cursor.fetchall()]
19
20 def fill_appointments():
21     conn = mysql.connector.connect(**db_config)
22     cursor = conn.cursor()
23
24     try:
25         cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
26         cursor.execute("TRUNCATE TABLE appointment")
27         cursor.execute("ALTER TABLE appointment AUTO_INCREMENT = 1")
28         cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
29
30         pet_list = get_pet_list(conn)
31         service_ids = get_service_ids(conn)
32         admin_ids = get_admin_ids(conn)
33
34         appointments = generate_appointments(pet_list, service_ids, admin_ids)
35
36         for appointment in appointments:
37             cursor.execute(
38                 """INSERT INTO appointment
39                    (status, appointment_date, appointment_time, reason,
40                     conclusion, treatment, service_id, pet_id, owner_id, admin_id)
41                    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)"""",
42                 (appointment['status'], appointment['appointment_date'],
43                  appointment['appointment_time'], appointment['reason'],
44                  appointment['conclusion'], appointment['treatment'],
45                  appointment['service_id'], appointment['pet_id'],
46                  appointment['owner_id'], appointment['admin_id']))
47             )

```

```

48
49         conn.commit()
50         print(f"Добавлено {len(appointments)} приемов!")
51
52 if __name__ == "__main__":
53     fill_appointments()

```

Листинг 10. Заполнение таблицы Veterinarian

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_veterinarians
4
5 def fill_veterinarians():
6     conn = mysql.connector.connect(**db_config)
7     cursor = conn.cursor()
8
9     cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
10    cursor.execute("TRUNCATE TABLE veterinarian")
11    cursor.execute("ALTER TABLE veterinarian AUTO_INCREMENT = 1")
12    cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
13
14    vets = generate_veterinarians(50)
15    for vet in vets:
16        cursor.execute(
17            """INSERT INTO veterinarian
18               (first_name, second_name, middle_name, phone_number, email, specialization,
19                qualification
19            VALUES (%s, %s, %s, %s, %s, %s, %s)"""",
20            (vet['first_name'], vet['second_name'], vet['middle_name'],
21            vet['phone_number'], vet['email'], vet['specialization'], vet['qualification
22            '])
23        )
24    conn.commit()
25    cursor.close()
26    conn.close()
27    print(f"Добавлено {len(vets)} ветеринаров!")
28
29 if __name__ == "__main__":
30     fill_veterinarians()

```

Листинг 11. Заполнение таблицы Veterinarian and appointment

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_vet_appointments
4
5 def get_appointment_ids(conn):
6     cursor = conn.cursor()
7     cursor.execute("SELECT appointment_id FROM appointment")
8     return [row[0] for row in cursor.fetchall()]
9
10 def get_vet_ids(conn):
11     cursor = conn.cursor()
12     cursor.execute("SELECT vet_id FROM veterinarian")
13     return [row[0] for row in cursor.fetchall()]
14
15 def fill_vet_appointments():
16     conn = mysql.connector.connect(**db_config)
17     cursor = conn.cursor()
18
19     try:
20         cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
21         cursor.execute("TRUNCATE TABLE veterinarian_appointment")
22         cursor.execute("ALTER TABLE veterinarian_appointment AUTO_INCREMENT = 1")
23         cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
24
25         appointment_ids = get_appointment_ids(conn)
26         vet_ids = get_vet_ids(conn)

```

```

27
28     vet_appointments = generate_vet_appointments(appointment_ids, vet_ids)
29
30     for va in vet_appointments:
31         cursor.execute(
32             """INSERT INTO veterinarian_appointment
33                (appointment_id, vet_id)
34                VALUES (%s, %s)"""",
35             (va['appointment_id'], va['vet_id'])
36         )
37
38     conn.commit()
39     print(f"Добавлено {len(vet_appointments)} связей ветеринаров с приемами!")
40
41 if __name__ == "__main__":
42     fill_vet_appointments()

```

Листинг 12. Заполнение таблицы Supplier

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_suppliers
4
5 def fill_suppliers():
6     suppliers = generate_suppliers(50)
7     conn = mysql.connector.connect(**db_config)
8     cursor = conn.cursor()
9
10    for supplier in suppliers:
11        cursor.execute(
12            "INSERT INTO supplier (name) VALUES (%s)",
13            (supplier['name'],)
14        )
15
16    conn.commit()
17    cursor.close()
18    conn.close()
19    print(f"Добавлено {len(suppliers)} поставщиков!")
20
21 if __name__ == "__main__":
22     fill_suppliers()

```

Листинг 13. Заполнение таблицы Product

```

1 import mysql.connector
2 import random
3 from config import db_config
4 from fake_data import generate_products
5
6 def get_supplier_ids(conn):
7     cursor = conn.cursor()
8     cursor.execute("SELECT supplier_id FROM supplier")
9     supplier_ids = [row[0] for row in cursor.fetchall()]
10    cursor.close()
11    return supplier_ids
12
13 def fill_products():
14     conn = mysql.connector.connect(**db_config)
15     cursor = conn.cursor()
16
17     cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
18     cursor.execute("TRUNCATE TABLE product")
19     cursor.execute("ALTER TABLE product AUTO_INCREMENT = 1")
20     cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
21
22     supplier_ids = get_supplier_ids(conn)
23
24     products = generate_products(supplier_ids, 700)
25     for product in products:

```

```

26         cursor.execute(
27             """INSERT INTO product
28                 (name, country, price, supplier_id)
29                 VALUES (%s, %s, %s, %s)""",
30             (product['name'], product['country'],
31             product['price'], product['supplier_id'])
32         )
33
34     conn.commit()
35     cursor.close()
36     conn.close()
37     print(f"Добавлено {len(products)} товаров!")
38
39 if __name__ == "__main__":
40     fill_products()

```

Листинг 14. Заполнение таблицы Purchase

```

1 import mysql.connector
2 from config import db_config
3 from fake_data import generate_purchases
4
5 def get_owner_ids(conn):
6     cursor = conn.cursor()
7     cursor.execute("SELECT owner_id FROM owners")
8     return [row[0] for row in cursor.fetchall()]
9
10 def get_product_ids(conn):
11     cursor = conn.cursor()
12     cursor.execute("SELECT product_id FROM product")
13     return [row[0] for row in cursor.fetchall()]
14
15 def get_admin_ids(conn):
16     cursor = conn.cursor()
17     cursor.execute("SELECT admin_id FROM administrator")
18     return [row[0] for row in cursor.fetchall()]
19
20 def fill_purchases():
21     conn = mysql.connector.connect(**db_config)
22     cursor = conn.cursor()
23
24     try:
25         cursor.execute("SET FOREIGN_KEY_CHECKS = 0")
26         cursor.execute("TRUNCATE TABLE purchase")
27         cursor.execute("ALTER TABLE purchase AUTO_INCREMENT = 1")
28         cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
29
30         owner_ids = get_owner_ids(conn)
31         product_ids = get_product_ids(conn)
32         admin_ids = get_admin_ids(conn)
33
34         purchases = generate_purchases(owner_ids, product_ids, admin_ids)
35
36         for purchase in purchases:
37             cursor.execute(
38                 """INSERT INTO purchase
39                     (purchase_date, product_id, owner_id, admin_id)
40                     VALUES (%s, %s, %s, %s)""",
41                 (purchase['purchase_date'], purchase['product_id'],
42                 purchase['owner_id'], purchase['admin_id'])
43             )
44
45         conn.commit()
46         print(f"Добавлено {len(purchases)} покупок!")
47
48 if __name__ == "__main__":
49     fill_purchases()

```
